國立中正大學

數學系統計科學所碩士班碩士論文

區間設限資料下基於樹狀方法分析

Tree-Based Analysis Under Interval Censored Data

研究生: 鄭景鴻 撰

指導教授: 謝進見 博士

民國 114 年 7 月

致謝詞

兩年的研究生涯匆匆就迎來了結尾，在這段時間發生了非常多的事情還有得到許多人的幫助。首先要先感謝在這一年中指導我們的謝進見教授，每個禮拜的meeting都給了我們許多的靈感和幫助。當我們遇到了不理解的地方老師都會歡迎我們用各種形式隨時發問，這大大加速了我們論文完成的進度。除此之外老師總是細心的檢視我們每個禮拜的結果，使得我們可以盡早將錯誤修正。除此之外我還要感謝我的研究好夥伴丞翔跟景茵。有他們的陪伴讓原本艱苦的研究生涯變的歡樂，我們經常在空閒時間討論各自的研究，彼此激發靈感。有好幾次的突破都是在歡樂的討論後突然靈光乍現的。沒有他們我沒辦法這麼順利的完成我的研究，也希望我也有爲他們帶來幫助。再來也感謝我們的潘宏裕教授和王价輝教授抽空來審查我們的論文，並幫助完善論文。獨自離鄉背井的來外地求學，我最掛念也最感謝的是我的家人，我的父母們。即使我已經長大了，他們依然幫我負擔學雜費以及生活費，讓我可以更加專注在統計的學習跟論文主題的研究。再來我還要謝謝我的朋友俊謙跟君瑋當我陷入低潮時，他們總是會當我堅實後盾並且給我諸多的建議。不僅限於情緒上的建議還有工作上的建議。這些鼓勵多次帶我脫離低谷快速地回歸正軌。研究所兩年的生涯很快也很快樂，回憶起來每個精彩時刻都對我意義重大，也讓我們同學們之間的關係更加的緊密，學弟妹也都很貼心友善，希望所有人都可以有順利且豐富的研究生涯。最後，希望所有未來要念研究所的莘莘學子們能夠面對困難、勇於挑戰陌生的事物、培養出堅強的精神力，最終帶著豐碩的成果離開研究所。

鄭景鴻 謹誌于

國立中正大學 數學系統計科學研究所

中華民國一一四年七月

摘要

本文專注於區間設限資料下的存活模型，提出了兩種新的估計方法，用於預測病患的存活時間。方法一融合XGBoost和廣義加法模型的概念估計出區間設限時間以及方法二是簡單的利用中點插補區間設限時間。此外，我們利用分配估計的方式估計右設限時間，並將其整合到樹模型中進行訓練和預測。在模擬實驗中，我們使用到基於Generalized Accelerated Failure Time (GAFT) 模型的框架構建出線性轉換、Box-Cox轉換、AFT模型和PH模型這幾種常見的存活分析模型，另外我們還自行設計一個不符合GAFT結構的模型。通過估計程序，得出新資料，並建立各類樹模型進行預測。隨後，通過模擬實驗，檢驗有效性並和其他區間設限樹方法進行比較。最後，我們應用這兩種新方法來分析真實數據並提出我們的解釋。

關鍵字：AFT模型, GAFT模型, 廣義加法模型, PH模型, XGBoost, 區間設限資料, 生存樹模型, 樹模型。

Abstract

This study focuses on survival modeling under interval-censored data and proposes two novel estimation approaches for predicting patient survival times. The first method integrates the concepts of XGBoost and Generalized Additive Models (GAM) to estimate interval-censored times, while the second method adopts a simpler strategy by imputing censored intervals using their midpoints. Additionally, we estimate right-censored times through distribution-based approaches and incorporate these estimates into tree-based models for training and prediction.

Within the framework of the Generalized Accelerated Failure Time (GAFT) model, we construct several classical survival models in simulation studies including the linear transformation model, the Box-Cox transformation model, the Accelerated Failure Time (AFT) model, and the Proportional Hazards (PH) model. We also design an additional model that intentionally violates the GAFT structure to evaluate robustness. Using the estimated data, we train a variety of tree-based models for survival prediction. Simulation experiments are conducted to assess the effectiveness of the proposed methods and compare them with existing interval-censored tree approaches. Finally, we apply both proposed methods to a real-world dataset and provide interpretations based on the results.

Keywords：AFT model, GAFT model, Generalized Additive Model, PH model, XGBoost, interval-censored data, survival tree models, tree-based methods.

Contents

1 Introduction **1**

2 Data and Model **3**

2.1 Interval-Censored Data 3

2.2 Generalized Accelerated Failure Time Model 4

2.2.1 Accelerated Failure Time Model 5

2.2.2 Cox Proportional Hazards Model 5

3 Review of Literature **7**

3.1 Basic Tree 7

3.2 Classification and Regression Trees 8

3.3 Ensemble Learning 9

3.3.1 Bagging 9

3.3.2 Random Forests 10

3.3.3 Boosting 11

3.3.4 Bayesian Additive Regression Trees 12

3.4 eXtreme Gradient Boosting 13

3.5 Survival Tree 14

3.6 IC Tree 15

4 Estimation Method **17**

4.1 Generalized Additive Model 17

4.2 Imputation Process 18

4.2.1 Interval-censored XGBoost-GAM Imputation Method 19

4.2.2 Midpoint Imputation Method 22

4.2.3 Gamma Right-Censoring Imputation Algorithm and Tree Model 22

4.3 Method Selection 25

5 Simulation **27**

5.1 Performance Metrics 27

5.1.1 Concordance Statistic 27

5.1.2 Mean Square Error (MSE) 28

5.1.3 Mean Absolute Error (MAE) 28

5.2 Simulation Settings 29

5.2.1 Comparison of Methods 31

5.3 Model Selection 38

6 Real Data Analysis **73**

6.1 Data Description 73

6.2 Plausibility evaluation 74

6.3 Range of Predicted Times 75

6.4 Predicted Times across Different Subgroups 75

7 Conclusion **84**

Reference **86**

Chapter 1

Introduction

Survival analysis is a commonly used statistical method, often applied in studies related to diseases, clinical trials, and other medical research. The outcome variable of interest is the time until a specific event occurs, such as death, disease onset, or relapse after recovery. However, in practice, it is often difficult to fully collect the exact survival time. Only partial information may be available, and this type of data is referred to as censored data. Since relying solely on the interval midpoint for rough imputation in interval-censored data may lead to biased results, which can ultimately affect the overall conclusions of the study, we aim to reduce such bias by using imputation and tree-based models to obtain more accurate predicts of event times. This study focuses on the estimation of outcomes in interval-censored data. To address the issue of incomplete data due to censoring, we propose two different imputation methods. The estimation approaches are applied separately to interval-censored and right-censored data. For interval-censored data, we present two methods: the first combines XGBoost with a non-parametric Generalized Additive Model (GAM), and the second is a simpler approach that imputes the midpoint of the censoring interval. For right-censored data, we handle it using a distribution-based estimation method. Once the data is completed, we employ

multiple tree-based models to predict survival times. To evaluate the effectiveness and feasibility of our method, we use ICtree, a well-known estimation method for interval-censored data, as a benchmark for comparison. In addition, our study proposes a model selection mechanism that can automatically choose the most suitable tree model for predicting survival time. The structure of this thesis is as follows: Chapter 2 introduces the concept of interval-censored data, the Generalized Accelerated Failure Time (GAFT) model, and other commonly used models for censored data in survival analysis. Chapter 3 provides an overview of various tree-based models, ensemble learning techniques, existing survival trees, and interval estimation trees, along with guidance on implementing them in R. Chapter 4 presents our proposed estimation methods and workflow, including two approaches for interval-censored data and one for right-censored data, along with detailed algorithms. This chapter also includes the design of the model selection strategy. Chapter 5 covers simulation experiments. We evaluate the performance of our proposed methods under various censoring scenarios, including different interval lengths and censoring proportions, and compare them with existing approaches. Chapter 6 presents an application to a real dataset involving the estimation of permanent tooth eruption times in elementary school students, demonstrating the practical performance of the time estimation. Finally, Chapter 7 summarizes the research findings, offers recommendations, and discusses potential areas for future improvement.

Chapter 2

Data and Model

2.1 Interval-Censored Data

In survival analysis, in addition to the commonly used right-censored models, another important type of data is interval-censored data. In real-world scenarios, patient conditions are typically monitored at regular follow-up intervals. As a result, the exact time of the event occurrence is often unknown, and researchers can only determine that the event occurred between two observation time points. Let y_i represent the true (unobserved) event time for the i th subject. What we observe in practice is a pair of interval endpoints (L_i,R_i), such that the event time lies within:

$$L_i < y_i < R_i,$$

Here, L_i denotes the last time the event had not yet occurred, and R_i indicates the first time the event was confirmed to have occurred. Thus, the pair (L_i,R_i) is the observed interval-censored data. If the event time has not occurred during the study period such data are considered right-censored, and the interval pair can be denoted as (L_i, ∞). We define

$$d_i = I[R_i < \infty],$$

where $I[\cdot]$ is the indicator function. Therefore, when $d_i = 1$, the observed interval is (L_i, R_i); and when $d_i = 0$, the interval becomes (L_i, ∞)

2.2 Generalized Accelerated Failure Time Model

According to Ridder (1990), the Generalized Accelerated Failure Time (GAFT) model is expressed as

$$h(Y_i) = X_i'\beta + \epsilon_i, i = 1, ..., n, \tag{2.1}$$

where Y_i is the dependent variable, and X_i' is the vector of observed covariates. The error term ϵ_i has an unknown distribution, which can be arbitrary. The parameter vector β is to be estimated. The function $h(\cdot)$ is assumed to be strictly monotonic, This model generalizes the classical Accelerated Failure Time (AFT) model by allowing both the transformation function $h(\cdot)$ and the error distribution to be unspecified. As a result, it is capable of capturing a broader range of survival patterns and covariate effects. The model can be equivalently rewritten as Thus, we can derive $Y_i \equiv h^{-1}(X_i'\beta + \epsilon_i)$

This highlights that the observed survival time Y_i is a transformed linear function of covariates plus a random error.

Importantly, both the widely used Cox proportional hazards (PH) model and the traditional Accelerated Failure Time (AFT) model can be seen as special cases within the GAFT framework. For instance

- When $h(y) = \log(y)$ the model reduces to the standard AFT form.
- When error tem follows the Gumbel extreme alue distribution, it aligns with the Cox model structure.

Thus, the GAFT model serves as a unifying and flexible platform for survival analysis, accommodating diverse data structures and relaxing restrictive distributional assumptions.

2.2.1 Accelerated Failure Time Model

The Accelerated Failure Time (AFT) introduced in Kalbfleisch and Prentice (1980) discussed about the estimation of semi-parametric models. The model is a widely adopted technique in survival analysis, used to assess how covariates influence the duration until a particular event takes place. Unlike the Cox model, which models the hazard function, the AFT model focuses directly on the survival time and assumes that covariates affect the timing of the event in a multiplicative fashion on the time scale. In essence, explanatory variables either speed up or slow down the occurrence of the event, effectively stretching or compressing the time axis. This makes the AFT model particularly intuitive for interpreting covariate effects. The AFT model is typically written as

$$\log(Y) = X'\beta_{+}\epsilon$$

2.2.2 Cox Proportional Hazards Model

The Cox Proportional Hazards Model, introduced by Cox (1972), is one of the most widely used regression models in survival analysis. It is a semi-parametric model that stands out for its flexibility—it does not require any specific distributional assumption for the baseline hazard function. Instead, it focuses solely on modeling the relationship between the explanatory variables and the hazard of an event. This feature allows the model to estimate the effect of covariates on the relative risk without having to specify the exact shape of the underlying hazard, making

it a popular choice in fields such as medicine, biostatistics, and epidemiology. The Cox PH model can be expressed as:

$$h(Y) = X'\beta + \epsilon, \tag{2.2}$$

where error term follows the Gumbel extreme value distribution.

Chapter 3

Review of Literature

3.1 Basic Tree

The concept of decision tree was first introduced by Morgan and Sonquist (1963) as a tree-structured model for handling both classification and regression problems. Later, the CART (Classification and Regression Trees) methodology, developed by Breiman et al. (1984), significantly advanced the use of decision trees and led to their widespread adoption in the field of machine learning. One of the key strengths of decision trees lies in their intuitive structure and ease of interpretation, closely mirroring the way humans reason through decisions. The basic idea is to partition the feature space into several subregions, where predictions are made based on either the mean (for regression) or the mode (for classification) of the training data within each region. Owing to this hierarchical and rule-based structure, the method is referred to as a "decision tree". We can implement a regression tree using the **tree()** function from the package **tree** in R language.

3.2 Classification and Regression Trees

The process of building a regression tree is known as recursive binary splitting, where the goal is to iteratively divide the data in a way that minimizes the Residual Sum of Squares (RSS). Starting from the root node, the algorithm splits the data step by step, each time creating two branches. This results in partitioning the predictor space into multiple distinct regions. This approach is typically considered a greedy algorithm, as it selects the best possible split at each step based on the current state, without evaluating the global optimality of the entire tree. During each split, the algorithm evaluates all possible features and split points to find the one that yields the lowest RSS. For a response variable $Y = (y_1, y_2, \ldots\ldots y_n)$, the feature space may be split into two subsets based on a threshold s for variable $X_k : \{X|X_k < s\}$ and $\{X|X_k \geq s\}$ The predicted value for any observation X_i is then assigned based on the mean (for regression) or mode (for classification) of the training samples within its corresponding region. RSS given by

$$\sum_{k=1}^{K} \sum_{i \in R_k} (y_i - \hat{y}_{R_k})^2, \tag{3.1}$$

where $\hat{y}_{R_k}$ denotes the mean response for the training dataset within the kth region. This splitting process is repeated iteratively, with new splits occurring only within the regions generated in the previous steps. The algorithm continues to grow the tree until it reaches a predefined stopping criterion, such as a minimum node size, insufficient information gain, or when no further meaningful splits can be made. We now shift our focus to classification problems. Unlike regression trees, which predict quantitative outcomes, classification trees are used when the response variable is qualitative—that is, the goal is to assign observations to discrete categories rather than predict numerical values. In classification trees, each observation is predicted as the most common class among the training samples in the same terminal node. The results are typically analyzed by examining the class prediction at each terminal

node and the distribution of classes within that node. Because the response variable is categorical, we cannot use RSS as the splitting criterion. Instead, we typically use measures like the Gini Index and Entropy as splitting metrics .

Gini index is a measure of total variance across the classes, defined by:

$$G = \sum_{t=1}^{T} \hat{p}_{mt}(1 - \hat{p}_{mt}), \tag{3.2}$$

where $\hat{p}_{mt}$ indicates the relative frequency of training instances belonging to class t within region m . The Gini index measures the overall variation among categories.

Entropy (D)

$$D = -\sum_{t=1}^{T} \hat{p}_{mt} \log_2 \hat{p}_{mt}. \tag{3.3}$$

Entropy measures the uncertainty within a node; the purer the node (i.e., the more dominated by a single class), the lower the entropy

3.3 Ensemble Learning

Ensemble Learning is a method of systematically combining multiple weak learners to build a stronger, more powerful model. While each weak learner may have limited predictive ability, combining them through ensemble methods can significantly improve accuracy and stability. Classic ensemble methods include Bagging, Random Forests, Boosting, Bayesian Additive Regression Trees (BART), and eXtreme Gradient Boosting (XGB). These methods often rely on regression or classification tree as their base learners.

3.3.1 Bagging

Bagging (Bootstrap Aggregating) is an ensemble learning technique introduced by Breiman (1996) that improves model performance by reducing variance through

repeated resampling. It creates multiple bootstrap training datasets, trains a separate model on each, and aggregates their predictions by averaging for regression tasks or majority voting for classification to produce the final output. This method is particularly effective for high-variance models like decision trees, as it enhances prediction stability and reduces overfitting. By combining the outputs of multiple models trained on varied data subsets, Bagging increases both robustness and accuracy. However, while Bagging reduces correlation among models, its performance gain may be constrained if the original dataset is small, as the resulting bootstrap samples may not be sufficiently diverse. For a training set with n independent observations, Bagging generates T bootstrap samples to train T models, denoted as $\hat{f}_1(x), \hat{f}_2(x), ..., \hat{f}_T(x)$ .The final Bagging prediction is:

$$\hat{f}_{bagging}(x) = \frac{1}{T}\sum_{t=1}^{T} \hat{f}_t(x) \tag{3.4}$$

We can implement bagging using the **randomforest(y $\sim$ x , data)** function from the package **randomForest** in R language.

3.3.2 Random Forests

Random Forest is an ensemble learning method based on the Bagging framework, proposed by Breiman (2001). It solves classification and regression problems by combining multiple decision trees with randomized behavior. Random Forest improves upon Bagging by introducing an additional layer of randomness instead of using all features when making a split (as Bagging does), Random Forest selects a random subset of features for each split. This further reduces the correlation between trees, thereby enhancing model performance. In fact, if all features are used at every split, Random Forest becomes equivalent to Bagging. We can implement randomforest using the **randomforest(y $\sim$ x ,data , ntree)** function from the package **randomForest** in R language.

3.3.3 Boosting

Boosting is a powerful ensemble technique that builds a strong learner by sequentially training and combining multiple weak learners. It focuses on reducing bias by progressively correcting the errors made by the previous model, thus improving overall predictive accuracy. Boosting can be applied to both classification and regression problems. Because each learner is trained based on the errors of the previous one, a strong dependency exists between models in the sequence. Here, we focus on the Gradient Boosting method proposed by Friedman (2001). This approach, a specific implementation of boosting, aims to improve performance by fitting residuals step-by-step. Methodology: The initial model makes a first-round prediction. In each subsequent iteration, the algorithm computes the residuals (errors from the previous model) and fits a new learner to these residuals. The newly fitted learner helps correct the errors from the previous stage. A shrinkage parameter, called the learning rate λ, is introduced to control the contribution of each new tree to the overall model. The update rule is as follows:

$$\hat{F}_t(x) = \hat{F}_{t-1}(x) + \lambda \hat{f}_t(x) \tag{3.5}$$

where $\hat{F}_t(x)$ represents the ensemble model after t iterations, and $\hat{f}_t(x)$ denotes the newly fitted weak learner (typically a regression tree) at iteration t. A smaller λ slows down learning but tends to produce more stable models and reduce the risk of overfitting. Conversely, a large λ speeds up training but may reduce generalization ability, impacting predictive performance. We can implement boosting using the **gbm(y $\sim$ x , data)** function from the package **gbm** in R language.

3.3.4 Bayesian Additive Regression Trees

Bayesian Additive Regression Trees (BART) is a nonparametric regression and classification method that combines Bayesian inference with a decision tree framework. It was introduced by Chipman, George, and McCulloch (2010). The core idea of BART is to construct a model by summing the predictions from multiple simple regression trees. The model parameters and tree structures are inferred using Markov Chain Monte Carlo (MCMC) sampling techniques. Compared to traditional ensemble methods such as Bagging or Boosting, BART is unique in the way it updates models. During training, BART iteratively updates each tree individually. For a given tree, the algorithm first subtracts the contributions of all other trees from the observed responses, yielding partial residuals. Unlike Boosting, which fits new trees to the residuals, BART perturbs existing trees—either structurally or by modifying their terminal node predictions—to improve model fit. These perturbations include two main types:

1. Structural changes to the tree .

2. Adjustments to the predicted values at the terminal nodes.

Through such updates, BART explores a broader model space and avoids getting trapped in local optima too early in the training process. Because the early iterations of the MCMC sampling may not represent the target distribution well, BART typically designates an initial burn-in period, during which models are not used for final prediction. The final predictive estimate is computed as the average of the samples collected after the burn-in period, and credible intervals for predictions can also be derived. Overall, BART offers a flexible and robust modeling framework. It provides strong predictive performance and enables full Bayesian inference to quantify uncertainty. This makes it especially suitable for high-dimensional data

and problems with complex nonlinear relationships.We can implement BART using the **gbart(x , y , x.test)** function from the package **BART** in R language.

3.4 eXtreme Gradient Boosting

XGBoost (eXtreme Gradient Boosting) is an efficient and scalable algorithm based on Gradient Boosted Decision Trees (GBDT). It was originally proposed by Chen and Guestrin (2016) and has since been widely applied to prediction tasks such as classification, regression, and ranking. XGBoost is an improved version of the basic Gradient Boosting Machine (GBM) framework. By incorporating system-level optimizations and regularization techniques, it significantly enhances both computational efficiency and generalization performance while maintaining high prediction accuracy. XGBoost adopts the concept of an additive model, where a new regression tree is added at each iteration to learn the residual errors from the previous round. Its objective function combines a loss term with a regularization term to balance model fit and complexity, thereby reducing the risk of overfitting. The objective function of XGBoost is defined as

$$L(\phi) = \sum_{i=1}^{n} l(y_i, \hat{y}_i) + \sum_{k=1}^{K} \Omega(f_k), \tag{3.6}$$

Here, $l(y_i, \hat{y}_i)$is the loss function (e.g., squared error, logistic loss), and $\Omega(f_k)$ is the regularization term for the kth tree, which takes the following form

$$\Omega(f) = \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} \omega_j^2, \tag{3.7}$$

where T denotes the number of leaf nodes in the tree, ω_j is the prediction score on the jth leaf node, and γ, λ control model complexity and the strength of regularization, respectively. Since the objective function involves the model function itself (i.e., the structure of the trees), XGBoost uses a forward additive model to approximate the

optimal solution by incrementally adding trees in each iteration. The prediction at the tth iteration is defined as:

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t)-1} + f_t(x_i), \tag{3.8}$$

Where $\hat{y}_i^{(t)}$ is the prediction of ith instance at tth iteration, and $f_t(x_i)$ represents the base learner added at iteration t. The corresponding objective function at iteration t is:

$$L^{(t)} = \sum_{i=1}^{n} l(y_i, \hat{y}_i^{(t-1)}) + f_t(x_i) + \Omega(f_t) \tag{3.9}$$

We can implement XGB using the **xgboost()** function from the package **xgboost** in R language.

3.5 Survival Tree

In early research, the classification and regression tree algorithm proposed by Breiman et al. (1984) attracted significant academic attention due to its clear structure, strong logical reasoning, and high interpretability. This method effectively presents interactions between variables, making it highly valued in subsequent developments. Gordon and Olshen (1985) extended the concept of decision trees to the field of survival data and developed the "survival tree" framework. However, in survival data, the actual time of event occurrence is often incompletely observed due to censoring, which poses challenges for traditional splitting criteria. Therefore, it is necessary to develop splitting and evaluation methods specifically designed for censored data. To address such issues, subsequent research has proposed various splitting and pruning methods tailored for censored data. Commonly used splitting criteria are mainly divided into two categories: node distance metrics, which aim to enhance heterogeneity between nodes; and node purity metrics, which focus on increasing the homogeneity of samples within nodes (Zhou McArdle, 2015; Molinaro

et al., 2004). Representative distance metrics include the likelihood ratio statistic (Ciampi et al., 1987) and conditional inference permutation tests (Hothorn et al., 2006). Moreover, some studies have suggested using P-values as the stopping rule for tree growth—if no significant split is found, tree expansion should be terminated (Hothorn et al., 2006). In terms of algorithm applications, the R provides several tools for building survival trees. For example, Therneau and Atkinson. (2010) proposed an algorithm implemented in the rpart package, which is based on Martingale residuals. Another commonly used method is the conditional inference tree (ctree) algorithm proposed by Hothorn et al. (2010), which has been integrated into the party package in R language. These methods are widely used tree-based modeling approaches in survival analysis and are often used as baseline models for comparison in later research. We can implement ctree using the **ctree(y ∼ x ,data)** or **ctree(Surv(y,d) ∼ x ,data)** function from the package **party** and implement rpart using the **rpart(Surv(y,d) ∼ x ,data)** function from the package **rpart** in R language.

3.6 IC Tree

The IC Tree, proposed by Fu and Simonoff (2017), is a framework that integrates recursive partitioning into a rigorous permutation testing theory. In the widely used tree algorithm CART, the selection of the splitting variable and the split point occurs simultaneously. This design tends to favor attributes that offer more potential split points, leading to bias in variable selection. In contrast, the conditional inference tree algorithm developed by Hothorn et al.(2006) and colleagues first selects the splitting variable based on the conditional distribution under the null hypothesis of independence between the response variable and covariates. Once the splitting variable is selected, the split point is then determined using standard

criteria. This two-step approach helps to eliminate bias in split point selection, resulting in a model that is more fair and robust. For interval-censored data, which are difficult to model using traditional approaches, IC Tree provides a nonparametric and structured alternative. It can handle various types of censoring and truncation, and its ability to visualize prediction rules enhances the interpretability of the results. The ICtree() function, implemented in the LTRCtrees package, provides support for interval-censored data analysis in R. We can implement ICtree using the **ICtree(Surv(L , R , type="interval2") ∼ x , data)** function from the package **LTRCtree** in R language.

Chapter 4

Estimation Method

In this chapter, we introduce two imputation strategies developed in this study. The first approach method using XGBoost and Generalized Additive Models (GAM) for interval censored data imputation, along with a right-censored imputation based on the Gamma distribution. The second approach applies a midpoint-based interval imputation, also combined with a right-censored imputation using the Gamma distribution. After performing imputation, both methods are incorporated into various tree based algorithms for further analysis. Additionally, we have designed a selection mechanism that automatically determines the most appropriate tree based model to use for each imputed dataset.

4.1 Generalized Additive Model

The GAM, introduced by Hastie and Tibshirani (1986), is a flexible statistical framework designed to model nonlinear relationships between the response and explanatory variables. Unlike traditional linear models that assume fixed linear effects, GAM allows each predictor's effect to be represented by a nonparametric

smooth function, while preserving an additive form that ensures interpretability. The key advantage of GAM lies in its ability to reveal hidden nonlinear patterns within data, while allowing each variable's influence to be examined independently. This enhances both the transparency and practical usefulness of the model. Due to its flexibility, GAM has been widely adopted in fields such as biostatistics, epidemiology, actuarial science, and economics. The method is well-supported in practice through tools such as the mgcv package in R.

Assume we have n observations and m covariates $X = (x_1, x_2....x_m)$. A linear model can be expressed as:

$$E(Y|X) = \beta_0 + X'\beta \tag{4.1}$$

In contrast, a GAM replaces the linear predictors with smooth functions for each covariate, capturing nonlinear dependencies more effectively:

$$g(E(Y|X)) = s_0 + \sum_{j=1}^{m} s_j(X_j) \tag{4.2}$$

In this formulation, it is not necessary to transform or pre-select the covariates X. Here, $g(\cdot)$ denotes the link function, while each $s_j(\cdot)$ is a smooth, data-driven function estimated from the data without assuming a specific parametric form.

4.2 Imputation Process

Before estimating using tree-based models, we need to first perform imputation on the actual time Y. The imputation process is divided into two parts: one for the $d_i = 1$ complete interval and the other for the $d_i = 0$ right-censored part. For the $d_i = 1$ data, we have prepared two imputation methods: the XGBoost-GAM imputation method and the midpoint imputation method. The XGBoost-GAM imputation method is self-developed by us, and the process is more complex. It will

be detailed in Section 4.2.1. The midpoint imputation method, on the other hand, is much simpler and is one of the most common and intuitive imputation methods for interval-censored data. It will be introduced in Section 4.2.2. Finally, for right-censored data, we use the Gamma right-censored imputation method, where we draw candidates from a Gamma distribution and compute the average to obtain the final imputed value. In addition, during our research, we found that the sampling range for the candidates must be restricted, which improves the accuracy of the sampling. Detailed information will be provided in Section 4.2.3. The chapter also includes step-by-step procedures that lead to our final imputed values.

4.2.1 Interval-censored XGBoost-GAM Imputation Method

Since this step is relatively complex, we will describe it in detail using an algorithmic approach.

Algorithm Procedure

Step 1:

First, we use the `ic_sp(Surv(L, R, type = "interval2") ~ ., data, model = "ph")` function from the `icenReg` package to roughly estimate the model parameters β using the PH model, Bergman (2017) proposed. Then, the linear predictors are calculated by $X\hat{\beta}$. Since the algorithm incorporates a GAM model to capture nonlinear relationships between the survival time and the covariates, this step serves only as a preliminary step for the subsequent application of the GAM model.

Step 2:

This step is performed only for data with $d_i = 1$. We aim to generate initial imputed values for fully interval-censored data using a truncated normal distribution. For the initial setting of distribution parameters, we set the distribution mean as the

mean of the midpoint of the interval im calculated from all observations with $d_i = 1$, and set the standard deviation as $\frac{stt}{3}$. Dividing by 3 is the optimal adjustment result after testing different values, where

$$im = \frac{1}{n_I} \sum_{i=1}^{n_I} \frac{(L_i + R_i)}{2},$$

where, n_I is the number of interval-censored observations, and stt is the standard deviation of the set st. Each element st_i in the set st is defined as $st_i =$ **runif**$(1, L_i, R_i)$,$i = 1, \ldots, n_I$. This parameter configuration was determined to be optimal through multiple trials, and is used for setting parameters in the truncated normal distribution.

Step 3:

Using the truncated normal distribution along with the calculated mean and standard deviation, we generate p candidate values (here, $p = 20$) for each observation. The code is **rtruncnorm(p,**L_i**,**R_i**, mean, sd)**. The initial imputed value $\hat{t}_{t,i}$ for each observation is then obtained by calculating the average of these p candidate values.

Step 4:

Substitute $\hat{t}_{t,i}$ and the corresponding variables to generate the XGBoost model. Then, use the same model and covariate to predict the value $\hat{t}_{x,i}$. If the resulting $\hat{t}_{x,i}$ does not fall within the interval (L_i, R_i), then for that observation, the value $\hat{t}_{x,i}$ remains as $\hat{t}_{t,i}$.

Step 5:

Let $\hat{t}_{g,i}^{(0)} = \hat{t}_{x,i}$. Input $\hat{t}_{g,i}^{(s)}$ and the linear predictor $X\hat{\beta}$ obtained in Step 1 to generate the GAM model. The code is **gmodel = gam(**$\hat{t}_{g,i}^{(s)} \sim$ **s(**$X\hat{\beta}$**),data)**, and the new predict value $\hat{t}_{g,i}^{(s+1)}$ is **predict(gmodel,newdata=**x_i**)**. where s the sth update or iteration.

Step 6:

Check whether $\hat{t}_{g,i}^{(s)}$ falls within the interval (L_i, R_i). If it does not, then $\hat{t}_{g,i}^{(s)}$ retains the previous value $\hat{t}_{g,i}^{(s-1)}$. Formally,

$$\hat{t}_{g,i}^{(s)} = \begin{cases} \hat{t}_{g,i}^{(s-1)}, & \text{if } \hat{t}_{g,i}^{(s)} \notin (L_i, R_i), \\ \hat{t}_{g,i}^{(s)}, & \text{if } \hat{t}_{g,i}^{(s)} \in (L_i, R_i), \end{cases}$$

Step 7:

Repeat Step 5 and Step 6 to update $\hat{t}_{g,i}^{(s)}$ and verify the condition. Repeat this cycle until the convergence criterion is met or until the loop completes u iterations (here, $u = 300$). The convergence criterion is defined as:

$$\frac{1}{n_I} \sum_{i=1}^{n_I} \left| \hat{t}_{g,i}^{(s)} - \hat{t}_{g,i}^{(s-1)} \right| < 0.001.$$

Step 8:

Take all the final generated observation values $\hat{t}_{g,i}^{(s)}$, calculate their mean and standard deviation, and then re-enter them into Step 3. Repeat the cycle until the target number of iterations cd is reached (here, $cd = 30$), after which the loop terminates.

Step 9:

After completing the loop, the final generated $\hat{t}_{g,i}^{(s)}$ in the last iteration becomes the imputed value for each fully interval-censored observation, which is denoted as $\hat{t}_{\text{interval,i}}$. The set of imputed values is denoted as

$$\hat{T}_{\text{interval}} = \{\hat{t}_{\text{interval,i}},\ i = 1, \ldots, n_I\}.$$

4.2.2 Midpoint Imputation Method

Midpoint imputation is a simple and intuitive method of imputation. Since our data type does not contain the true time y_i, but instead the left and right endpoints of the interval in which the true time is located, we typically use the midpoint of these two endpoints as the imputed value $\hat{t}_{mid,i}$. The imputed value $\hat{t}_{mid,i}$ is defined as

$$\hat{t}_{mid,i} = \frac{1}{2}(L_i + R_i).$$

Thus, the set of midpoint imputed values is represented as

$$\hat{T}_{mid} = \{\hat{t}_{mid,i},\ i = 1, \ldots, n_I\}.$$

Since this method does not involve excessive assumptions, it is computationally simple and can handle cases where both endpoints exist effectively. This method is typically applied to datasets requiring imputation. Notably, when $d_i = 1$, this imputation method does not significantly impact subsequent calculations, whereas data with $d_i = 0$ will be processed separately.

4.2.3 Gamma Right-Censoring Imputation Algorithm and Tree Model

In the case of right-censored data, our approach is straightforward. Since the exact time information for right-censored observations is unknown, we utilize the gamma distribution to generate imputed values for each observation with $d_i = 0$. Here, we aim to predict time, so the exponential distribution is considered first, as it is a special case of the gamma distribution. Thus, we adjust the parameter settings to adopt the more flexible gamma distribution. The specific algorithmic procedure is as follows

Step 1:

We use the R package `fitdistrplus`, which proposed by Muller et al(2015), to estimate the distribution parameters for the left endpoints L_i of all training data. The function `fitdist(`$L + 10^{-10}$`, "gamma")`, where L=($L_1 \ldots$,L_n), is employed to estimate the parameters (α, λ). There is an extremely small value added to L_i ensures that L_i values are non-zero.

Step 2:

We incorporate the right-censoring rate r of the data to design a parameter adjustment factor $\frac{r}{1-r}$. The original parameters esrimations $(\hat{\alpha}, \hat{\lambda})$ are adjusted to $(\alpha_{new}, \lambda_{new})$. This adjustment is made based on the rationale that differences in right-censoring proportions may influence the sampling results. This adjustment preserves the mean of gamma distribution while adjusting the variance according to different censoring ratios.

The adjusted parameters are defined as

$$\alpha_{new} = \frac{r}{1-r} \times \hat{\alpha}, \quad \lambda_{new} = \frac{r}{1-r} \times \hat{\lambda}.$$

During our research, we also considered setting the adjustment factor as $\frac{1-r}{r}$, but this adjustment method performed worse in cases of light right-censoring and did not show better performance in cases of heavy right-censoring, even performing worse than the current adjustment method.

Step 3:

We randomly generate a set of candidate values vv from the gamma$(\alpha_{new}, \lambda_{new})$ distribution. From this set, we select 20 candidate values that fall within the reasonable range, and the candidate values set is defined as

$$vv_i = \{vv_{ij} \mid L_i \leq vv_{ij} \leq RL_i, j = 1, \ldots, 20\}, i = 1, \ldots, n_R.$$

The imputed right-censoring value $\hat{t}_{right}^{(i)}$ for each observation is then calculated as the average of these selected candidate values

$$\hat{t}_{right,i} = \frac{1}{20}\sum_{j=1}^{20} vv_{ij}, \quad i = 1, \ldots, n_R,$$

where n_R is the number of right-censored observations. The set of imputed right-censoring values is defined as

$$\hat{T}_{right} = \{\hat{t}_{right,i},\ i = 1, \ldots, n_R\}.$$

The upper limit RL_i for each sample is calculated based on the maximum interval length ma among all $d_i = 1$ observations

$$m_R = \max_{d_i=1}(R_i - L_i), i = 1 \ldots n_R$$

and

$$RL_i = L_i + (1.5 + r^2 \frac{\alpha_{new}}{\lambda_{new}}) \times m_R.$$

The upper limit RL_j is determined based on extensive trials and adjustments. The term $1.5m_R$ serves as the minimum guaranteed range length to ensure that the interval remains effective when the right-censoring rate is low. The term $r^2 \times \frac{\alpha_{new}}{\lambda_{new}}$ scales the range based on the censoring rate. This design provides better estimation accuracy in simulations compared to the scenario without an upper limit adjustment.

Step 4:

Combine all the imputed values $\hat{T}_{interval}$ from Section 4.2.1 with $\hat{T}_{right}$ to obtain the final imputed set $\hat{T}_{imputation}$

$$\hat{T}_{imputation} = \{\hat{t}_{interval,}, \hat{t}_{right,i}\}.$$

Similarly, combine the midpoint imputed values $\hat{T}_{mid}$ from Section 4.2.2 with $\hat{T}_{right}$ to obtain the final midpoint imputed set $\hat{T}_{middle}$

$$\hat{T}_{middle} = \{\hat{t}_{mid,i}, \hat{t}_{right,i}\}.$$

Step 5:

Fit the final imputed sets $\hat{T}_{imputation}$ and $\hat{T}_{middle}$ into each of the tree models (regression tree, bagging, random forest, boosting, XGBoost, BART, ctree, rpart) to train the models and obtain the predicted values for the test set in each method.

4.3 Method Selection

After different simulation studies, we found that there is no single method that performs best in all situations. To produce relatively reliable results that closely approach optimal outcomes, we developed a method selection approach. However, a major challenge in method selection arises because, in real-world scenarios, our data type makes it impossible to calculate MSE or any effectiveness metric. Since the true values y_i can not be observed, conventional MSE can not be directly calculated as a basis for evaluating predictive accuracy. Nevertheless, this does not mean that we cannot find a comparable alternative. Given that the imputed right-censoring values $\hat{t}_{right,i}$ have already considered the reasonable distribution within the original interval, we designate the elements in $\hat{T}_{imputation}$ as "pseudo-true values" for comparing each tree model method. For each tree model method, the predicted values are denoted as $\hat{t}_{method,i}$, where $method$ = {regression tree, bagging, boosting, random forest, XGBoost, BART, ctree, rpart}. The MSE for each method is calculated by replacing the true values with the pseudo-true values $\hat{t}_{imputation,i}$:

$$MSE_{method} = \frac{1}{n}\sum_{i=1}^{n}(\hat{t}_{method,i} - \hat{t}_{imputation,i})^2.$$

The method with the lowest MSE_{method} is selected as the predictive method for the test dataset. Multiple simulations have confirmed that this method selection

approach is reliable.

Model Fitting Procedure

Step 1:

The set of imputed values $\hat{T}_{imputation}$ along with the training data are fed into each tree-based model to fit individual models.

Step 2:

The training data is then input into the fitted tree models to obtain the predicted values based on training data for each model:

- Regression Tree: $\hat{t}_{regression,i}$,
- Bagging: $\hat{t}_{bagging,i}$,
- Boosting: $\hat{t}_{boosting,i}$,
- Random Forest: $\hat{t}_{randomforest,i}$,
- XGBoost: $\hat{t}_{xgboost,i}$,
- BART: $\hat{t}_{BART,i}$,
- ctree: $\hat{t}_{ctree,i}$,
- rpart: $\hat{t}_{rpart,i}$.

Each $\hat{t}_{method,i}$ represents the predicted value using a specific tree-based model.

Step 3:

Calculate the MSE_{method} for each tree-based model and select the method with the minimum MSE_{method} ,where

$$MSE_{method} = \frac{1}{n}\sum_{i=1}^{n}(\hat{t}_{imputation,i} - \hat{t}_{method,i})^2.$$

Chapter 5

Simulation

5.1 Performance Metrics

5.1.1 Concordance Statistic

The Concordance Statistic, also referred to as C-index, was introduced by Harrell et al. (1982). This index is used to evaluate the concordance between the predicted survival times and the actual observed survival times. By comparing the predicted and actual survival values, the concordance is measured. As a result, the C-index provides a measure of model performance with a specific focus on ranking. The formula for the C-index is as follows

$$C = P(\hat{Y}_i > \hat{Y}_j | Y_i > Y_j),$$

where Y_i and Y_j represent the observed survival times, and $\hat{Y}_i$ and $\hat{Y}_j$ represent the predicted survival times. C-index is directional and used to evaluate the ranking of the predicted survival times in relation to the actual observed times. When the C-index value is 0.5, it indicates that the model performs similarly to random guessing. If the C-index value is 1, it implies perfect concordance between the predicted and

observed survival times. In addition to the C-index, Uno et al. (2011) proposed a modified version of this statistic that can be applied in non-parametric scenarios. In simulations, the calculation for the C-index is as follows:

$$C = \frac{\sum_{i,j}[y_i > y_j][\hat{y}_i > \hat{y}_j]}{\sum_{i,j}[y_i > y_j]},$$

where y_i and y_j are the observed survival times and $\hat{y}_i$ and $\hat{y}_j$ are predicted survival times. C-index is a directional statistic used to rank patients based on their predicted survival times. As such, the C-index serves as a vital tool for evaluating predictive model performance. However, it has limitations, and research may consider combining it with other indices for a more comprehensive evaluation.

5.1.2 Mean Square Error (MSE)

Mean Square Error (MSE) is one of the most commonly used performance evaluation metrics. The core idea behind MSE is to directly measure the squared differences between the predicted values and the observed values. Since it squares the errors, larger errors are penalized more, making MSE particularly sensitive to large deviations. This characteristic helps in emphasizing larger errors, which can heavily influence the model's performance. The formula for MSE is as follows

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(e_i)^2, where \quad e_i = (y_i - \hat{y}_i).$$

As shown, MSE squares the errors, which magnifies the impact of large deviations. This makes MSE , highlighting their influence on the model's performance.

5.1.3 Mean Absolute Error (MAE)

Mean Absolute Error (MAE) is similar to MSE in that it also measures the prediction error, but the calculation method differs. Instead of squaring the errors,

MAE calculates the absolute difference between the predicted and observed values. Therefore, MAE is less sensitive to large deviations compared to MSE. The formula for MAE is as follows:

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|e_i|, where \quad e_i = (y_i - \hat{y}_i).$$

Even if the errors are large, MAE does not penalize them as much as MSE, making MAE a more stable indicator that is less influenced by extreme errors.

5.2 Simulation Settings

We consider five different models and three different interval length settings to compare three types of right censoring: no censoring, mild censoring, and heavy censoring. The levels of right censoring are classified as follows

- No censoring, with a right censoring rate of 0%.
- Light censoring, with a right censoring rate of approximately 20%.
- Heavy censoring, with a right censoring rate of approximately 40%.

The structure of the intervals is generated by starting from 0 and adding a randomly generated interval length C, thereby obtaining $L_1, L_2, \ldots, L_k$, where:

$$L_i = L_{i-1} + C, \quad L_0 = 0.$$

When the true value y_i falls within the interval (L_i, L_{i+1}), the right endpoint R_i is defined as L_{i+1}. Thus, the observed interval endpoints (L_i, R_i) are determined. If the true value y_i exceeds L_k, falling within (L_k, ∞), the data is defined as right-censored.

The interval lengths are categorized into three levels:

- Small intervals: $C \sim U(0.3, 0.7)$.
- Medium intervals: $C \sim U(1, 4)$.
- Large intervals: $C \sim U(3, 7)$.

We adjust the degree of right censoring by controlling the number of intervals k. Model parameters are slightly adjusted based on interval length and censoring degree. we repeat the simulation 500 times under each situation. We also examine the impact of sample size and censoring on model performance. The predicted value of ICtree or ctree is taken as the median of the corresponding Kaplan-Meier curve, generated by the tree. When the tail of the corresponding Kaplan Meior curve is larger than 0.5, the predicted value would be ∞. The first ctree, shown under the ICtree section in the table, uses the midpoint method for interval imputation but does not apply any imputation for right-censored data. The code is written as: **ctree(Surv(y, d) ~ x, data)** The second ctree, shown under the "Midpoint" section, represents the method that combines midpoint-based interval imputation with Gamma-based right-censored imputation. The code is written as: **ctree(y ~ x, data)** The third ctree, shown under the "XGB-GAM" section, represents the method that combines XGB-GAM-based interval imputation with Gamma-based right-censored imputation. The code is written as: **ctree(y ~ x, data)**. Here, y is the imputations by the corresponding method. Since the results are similar, most of the tables are included in the appendix.

5.2.1 Comparison of Methods

1. Linear Model

The data in this model is generated under the framework of a GAM, specifically using the GAFT model structure. This model includes four covariates plus a random error term ϵ. The covariates are generated as follows

- $X_1 \sim \mathcal{N}(4.6, 1)$,
- X_2 discrete uniform on 0,1,2,3 ,
- $X_3 \sim$ Poisson(3),
- $X_4 \sim$ Binomial$(2, 0.5)$,
- $\epsilon \sim \mathcal{N}(0, 1)$.

The specific model form is

$$Y_i = \beta_1 X_{1i} + \beta_2 X_{2i} + \beta_3 X_{3i} + \beta_4 X_{4i} + \epsilon_i.$$

Under different interval length scenarios

Small Interval :

$$Y_i = 0.5X_{1i} + 0.55X_{2i} + 0.8X_{3i} + 0.3X_{4i} + \epsilon_i$$

Medium Interval :

$$Y_i = 2.5X_{1i} + 2.55X_{2i} + 1.8X_{3i} + 1.3X_{4i} + \epsilon_i.$$

Large Interval :

$$Y_i = 5.5X_{1i} + 5.55X_{2i} + 5.8X_{3i} + 5.3X_{4i} + \epsilon_i.$$

The simulation results with 500 repetitions, as referenced in Table 5.1 and Table 5.2 for small interval lengths, Table 5.3 and Table 5.4 for medium interval lengths, and Table 5.5 and Table 5.6 for large interval lengths, indicate that the best performing algorithm in all scenarios is BART, based on our imputation methods. Another noteworthy observation is that the results of both methods (XGB-GAM imputation and Middle imputation) are quite similar. However, under the no-censoring scenario, the XGB-GAM imputation method slightly outperforms the Middle imputation method. Nevertheless, as the censoring proportion increases, the advantage of XGB-GAM gradually diminishes. Both of these imputation methods demonstrate superior performance compared to the ICtree method and the ctree method, which are mentioned in the referenced literature. This superiority is evident across all three censoring levels. The ICtree method estimates the values through survival curves. Thus, when any K-M curve fails to descend to 0.5, the estimated values are likely to be overestimated. This explains why, as the censoring proportion increases, the effectiveness indicators (MSE and MAE) surge significantly. However, the C-index is not affected by overestimation of the estimated values. In both light-censoring and heavy-censoring scenarios, our two methods consistently yield better C-index results without encountering issues of estimation failure.

2. Box-Cox Transformation Model

This model is also based on the GAFT framework, with the variable settings for the Box-Cox transformation identical to the linear model above. The error term ϵ_i is drawn from a normal distribution with a mean of 3 and a standard deviation of 0.5. The model is written as

$$Y_i = \left(\frac{1}{2}(\beta_1 X_{1i} + \beta_2 X_{2i} + \beta_3 X_{3i} + \beta_4 X_{4i} + \epsilon_i) + 1\right)^2.$$

Small Interval :

$$Y_i = \left(\frac{1}{2}(0.5X_{1i} + 0.55X_{2i} + 0.8X_{3i} + 0.3X_{4i} + \epsilon_i) + 1\right)^2.$$

Medium Interval :

$$Y_i = \left(\frac{1}{2}(2.5X_{1i} + 2.55X_{2i} + 1.8X_{3i} + 1.3X_{4i} + \epsilon_i) + 1\right)^2.$$

Large Interval :

$$Y_i = \left(\frac{1}{2}(5.5X_{1i} + 5.55X_{2i} + 5.8X_{3i} + 5.3X_{4i} + \epsilon_i) + 1\right)^2.$$

The simulation results with 500 repetitions, as referenced in Table 5.7 and Table 5.8 for small interval lengths, Table 5.9 and Table 5.10 for medium interval lengths, and Table 5.11 and Table 5.12 for large interval lengths, indicate that the best performing algorithm in all scenarios is BART, based on our imputation methods. Focusing on the comparison between the XGBoost-GAM and Middle imputation methods, we observe that although the results are close, the Middle method slightly outperforms the XGBoost-GAM method in most cases.

3. PH Model

This dataset is generated based on the proportional hazards (PH) model. The error term ϵ_i is drawn from a Gumbel extreme value distribution. The transformation function is defined as

$$h^{-1}(x) = \log(\exp(x) + 1).$$

This model includes four covariates plus a random error term . The covariates are generated as follows

- $X_1 \sim \mathcal{N}(4.6, 1)$,

- $X_2 \sim \text{Gamma}(3, \lambda = 3)$,
- $X_3 \sim \text{Poisson}(3)$,
- $X_4 \sim \text{Binomial}(2, 0.5)$,
- $\epsilon \sim$ Gumbel extreme value distribution .

The model is expressed as

$$Y_i = \log(\exp(\beta_1 X_{1i} + \beta_2 X_{2i} + \beta_3 X_{3i} + \beta_4 X_{4i} + \epsilon_i) + 1).$$

Small Interval :

$$Y_i = \log(\exp(0.5X_{1i} + 0.55X_{2i} + 0.8X_{3i} + 0.3X_{4i} + \epsilon_i) + 1).$$

Medium Interval :

$$y_i = \log(\exp(2.5X_{1i} + 3X_{2i} + 1.8X_{3i} + 1.5X_{4i} + \epsilon_i) + 1).$$

Large Interval :

$$y_i = \log(\exp(5.6X_{1i} + 6X_{2i} + 5.8X_{3i} + 5.5X_{4i} + \epsilon_i) + 1).$$

The simulation results with 500 repetitions, as referenced in Table 5.13 and Table 5.14 for small interval lengths, Table 5.15 and Table 5.16 for medium interval lengths, and Table 5.17 and Table 5.18 for large interval lengths. The best results for both the Middle and XGBoost-GAM imputation methods are consistently achieved by the BART algorithm. In the scenario no-censoring, the XGBoost-GAM imputation method slightly outperforms the Middle imputation method. However, as the proportion of right censoring and interval length increase, the advantage of XGBoost-GAM gradually diminishes. Both imputation methods demonstrate significantly superior performance across all three censoring proportions in comparison to the ICtree method mentioned in the literature and the ctree method using midpoint imputation, with more pronounced improvements observed in the C-index.

4. AFT Model

This dataset is generated based on the AFT (Accelerated Failure Time) model. The transformation function is defined as

$$h^{-1}(x) = \exp(x).$$

This model includes four covariates and a random error term ϵ_i. The covariates are generated as follows

- $X_1 \sim \text{Bernoulli}(0.5)$,
- $X_2 \sim \mathcal{N}(1.3, 0.3)$,
- $X_3 \sim \text{Gamma}(3, \lambda = 10)$,
- $X_4 \sim \text{Beta}(1.5, 0.5)$,
- $\epsilon_i \sim \mathcal{N}(0.8, 0.3)$,

The model form is

$$Y_i = \exp(\beta_1 X_{1i} + \beta_2 X_{2i} + \beta_3 X_{3i} + \beta_4 X_{4i} + \epsilon_i).$$

This model is typically used when covariates are believed to have multiplicative effects on the outcome. As it often produces wider data ranges, we use it as a robustness stress test.

Small Interval :

$$Y_i = \exp(0.8X_{1i} + 0.5X_{2i} + 0.6X_{3i} + 0.3X_{4i} + \epsilon_i).$$

Medium Interval :

$$Y_i = \exp(X_{1i} + 0.5X_{2i} + 0.75X_{3i} + 0.7X_{4i} + \epsilon_i).$$

Large Interval :

$$Y_i = \exp(1.5X_{1i} + 0.5X_{2i} + 0.8X_{3i} + 0.78X_{4i} + \epsilon_i).$$

The simulation results with 500 repetitions, as referenced in Table 5.19 and Table 5.20 for small interval lengths, Table 5.21 and Table 5.22 for medium interval lengths, and Table 5.23 and Table 5.24 for large interval lengths, indicate that the best performing algorithm in all scenarios is BART, based on our imputation methods. Although the performance of ICtree and ctree under the no-censoring scenario is already comparable to some models in our proposed interpolation methods, the BART model under our interpolation methods still outperforms them. Additionally, the differences between the XGBoost-GAM and Middle methods are not particularly significant. Specifically, under the no-censoring scenario, the Middle imputation method performs slightly better than XGBoost-GAM imputation method. However, one noteworthy observation is that, under certain tree-based models, the MSE values under light censoring are smaller than those under no censoring. We attribute this to the characteristics of the GAMMA distribution in the right censoring imputation. Since the AFT model applies the GAFT structure in the exponential term, the distribution might be more closely approximated by the GAMMA distribution, leading to imputation values under light censoring being closer to the true values. Nevertheless, as the censoring level increases to heavy censoring, the amount of observable data decreases, making accurate estimation more difficult. Therefore, this peculiar phenomenon only appears under light censoring and is likely a rare coincidence that corrects itself with larger sample sizes. Notably, under the strongest BART model, this anomaly is not as pronounced.

5. Weibull Model

This dataset is designed by us which is not the GAFT model. The generation mechanism assumes that an event is triggered by the earliest among multiple independent failure sources, which means the event time follows the minimum of several Weibull distributions. Each covariate is drawn from a Weibull distribution with specific parameters

- $X_1 \sim \text{Weibull}(0.6, 35)$,
- $X_2 \sim \text{Weibull}(0.9, 56)$,
- $X_3 \sim \text{Weibull}(2.6, 43)$,
- $X_4 \sim \text{Weibull}(1.2, 40)$,
- $\epsilon_i \sim \mathcal{N}(3, 0.5)$.

The model is expressed as

$$Y_i = \min(\beta_1 X_{1i}, \beta_2 X_{2i}, \beta_3 X_{3i}, \beta_4 X_{4i}) + \epsilon_i.$$

Small Interval :

$$Y_i = \min(0.5X_{1i}, 0.55X_{2i}, 0.8X_{3i}, 0.3X_{4i}) + \epsilon_i.$$

Medium Interval :

$$Y_i = \min(2.5X_{1i}, 2.55X_{2i}, 1.8X_{3i}, 1.3X_{4i}) + \epsilon_i.$$

Large Interval :

$$Y_i = \min(3.5X_{1i}, 3.55X_{2i}, 2.8X_{3i}, 1.3X_{4i}) + \epsilon_i.$$

The simulation results with 500 repetitions, as referenced in Table 5.25 and Table 5.26 for small interval lengths, Table 5.27 and Table 5.28 for medium interval lengths, and Table 5.29 and Table 5.30 for large interval lengths. The XGBoost method under our interpolation methods achieves the best performance. Unlike the previous simulation results, BART not only fails to be the best-performing method but also shows relatively weak competitiveness. The unique results observed in the Weibull model prompted us to develop a method selection mechanism to automatically identify the strongest tree model for data processing. As observed in the tables, ICtree is notably weak under this complex model, while our imputation methods consistently outperform ICtree and ctree in all three performance metrics. As for the comparison between the Middle and XGBoost-GAM methods, the Middle method consistently performs better than XGBoost-GAM method in most cases.

5.3 Model Selection

Table 5.31, 5.32, and 5.33 present the model selection results under different censoring scenarios across all model configurations. The best or most dominant result is determined based on all the simulation settings presented in Section 5.2. "Correct" indicates the truly correct method at that level, while "Selected" refers to the method chosen by our selection procedure from section 4.3. These tables report only the selection proportions of the top three models, with proportions expressed in percentages. Ideally, the model selected by the selection mechanism should exactly match the best-performing model identified in the simulation results. As shown in the tables, the selection mechanism successfully identified the best model in all 45 scenarios, achieving perfect consistency with the simulation-based results. In some cases, however, the selection proportion for the best model was relatively low. In such situations, we further examine whether the second- and third-best

selected models were also among the most competitive tree-based models. After taking this into account, we find that in the vast majority of cases, the selected models include either the top-performing or second-best model in approximately 80% of the scenarios. This indicates that the proposed model selection procedure not only demonstrates very high accuracy in identifying the optimal model but also consistently selects strong alternatives when uncertainty exists.

Table 5.1: Linear model for Small Interval with n=200.

Method	no-censored			light-censored			heavy-censored		
	C-index	MSE	MAE	C-index	MSE	MAE	C-index	MSE	MAE
IC_tree	0.702	2.169	1.156	0.699	Inf	Inf	0.683	Inf	Inf
ctree	0.745	1.791	1.064	0.734	Inf	Inf	0.709	Inf	Inf
Middle									
regression	0.756	1.706	1.04	0.751	1.778	1.052	0.742	2.046	1.118
bagging	0.79	1.339	0.921	0.785	1.487	0.959	0.778	1.813	1.048
randomforest	0.756	1.316	0.91	0.751	1.483	0.955	0.742	1.825	1.049
boosting	0.76	1.829	1.073	0.755	1.827	1.067	0.741	2.062	1.126
xgboost	0.768	1.645	1.02	0.763	1.719	1.035	0.749	2.012	1.111
BART	0.805	1.179	0.861	0.804	1.284	0.891	0.798	1.603	0.985
ctree	0.75	1.723	1.044	0.745	1.793	1.062	0.738	2.065	1.128
rpart	0.752	1.776	1.056	0.743	Inf	Inf	0.72	Inf	Inf
XGB-GAM									
regression	0.757	1.704	1.04	0.751	1.778	1.053	0.742	2.041	1.117
bagging	0.79	1.337	0.92	0.785	1.486	0.958	0.778	1.813	1.048
randomforest	0.757	1.314	0.909	0.751	1.482	0.954	0.742	1.825	1.048
boosting	0.761	1.824	1.071	0.756	1.82	1.065	0.741	2.061	1.126
xgboost	0.768	1.646	1.02	0.764	1.719	1.034	0.749	2.007	1.109
BART	0.805	1.178	0.861	0.804	1.284	0.891	0.798	1.604	0.986
ctree	0.75	1.718	1.042	0.744	1.794	1.062	0.738	2.066	1.129
rpart	0.754	1.747	1.047	0.744	Inf	Inf	0.721	Inf	Inf

Table 5.2: Linear model for Small Interval with n=300.

Method	no-censored			light-censored			heavy-censored		
	C-index	MSE	MAE	C-index	MSE	MAE	C-index	MSE	MAE
IC_tree	0.719	2	1.117	0.719	Inf	Inf	0.702	Inf	Inf
ctree	0.756	1.658	1.026	0.747	Inf	Inf	0.722	Inf	Inf
Middle									
regression	0.763	1.61	1.01	0.76	1.696	1.028	0.751	1.951	1.092
bagging	0.793	1.297	0.905	0.789	1.443	0.945	0.782	1.745	1.028
randomforest	0.763	1.265	0.893	0.76	1.435	0.94	0.751	1.749	1.024
boosting	0.768	1.694	1.036	0.762	1.729	1.039	0.75	1.949	1.093
xgboost	0.771	1.594	1.003	0.765	1.688	1.027	0.753	1.945	1.091
BART	0.807	1.141	0.85	0.806	1.255	0.882	0.801	1.546	0.966
ctree	0.761	1.606	1.009	0.758	1.686	1.027	0.751	1.95	1.094
rpart	0.757	1.688	1.033	0.751	Inf	Inf	0.729	Inf	Inf
XGB-GAM									
regression	0.763	1.604	1.009	0.76	1.691	1.027	0.75	1.954	1.092
bagging	0.793	1.296	0.905	0.789	1.443	0.945	0.782	1.745	1.028
randomforest	0.763	1.263	0.892	0.76	1.435	0.939	0.75	1.749	1.024
boosting	0.768	1.691	1.034	0.763	1.725	1.037	0.75	1.951	1.093
xgboost	0.772	1.593	1.004	0.765	1.69	1.028	0.754	1.937	1.089
BART	0.807	1.141	0.85	0.806	1.255	0.882	0.801	1.546	0.966
ctree	0.761	1.604	1.008	0.758	1.687	1.027	0.751	1.949	1.094
rpart	0.761	1.66	1.024	0.753	Inf	Inf	0.729	Inf	Inf

Table 5.3: Linear model for Medium Interval with n=200.

Method	no-censored			light-censored			heavy-censored		
	C-index	MSE	MAE	C-index	MSE	MAE	C-index	MSE	MAE
IC_tree	0.753	11.239	2.651	0.741	Inf	Inf	0.72	Inf	Inf
ctree	0.775	9.762	2.472	0.759	Inf	Inf	0.736	Inf	Inf
Middle									
regression	0.795	8.395	2.29	0.791	9.409	2.415	0.782	10.826	2.547
bagging	0.888	3.445	1.429	0.882	4.362	1.568	0.875	6.422	1.822
randomforest	0.795	3.516	1.434	0.791	4.283	1.544	0.782	6.385	1.805
boosting	0.882	3.444	1.442	0.875	4.614	1.638	0.863	6.485	1.865
xgboost	0.884	3.287	1.409	0.875	4.525	1.61	0.863	6.562	1.866
BART	0.917	1.756	1.027	0.913	2.632	1.199	0.907	4.557	1.476
ctree	0.788	8.897	2.365	0.783	9.866	2.472	0.775	11.426	2.629
rpart	0.77	9.961	2.493	0.76	Inf	Inf	0.74	Inf	Inf
XGB-GAM									
regression	0.796	8.33	2.283	0.79	9.472	2.419	0.781	10.913	2.566
bagging	0.89	3.377	1.414	0.884	4.333	1.561	0.877	6.435	1.822
randomforest	0.796	3.482	1.426	0.79	4.271	1.541	0.781	6.414	1.81
boosting	0.889	3.098	1.363	0.88	4.361	1.58	0.867	6.322	1.829
xgboost	0.89	3.018	1.347	0.88	4.333	1.567	0.866	6.421	1.839
BART	0.918	1.7	1.013	0.914	2.626	1.197	0.907	4.594	1.485
ctree	0.789	8.825	2.357	0.783	9.836	2.467	0.774	11.463	2.635
rpart	0.785	9.189	2.392	0.772	Inf	Inf	0.749	Inf	Inf

Table 5.4: Linear model for Medium Interval with n=300.

Method	no-censored			light-censored			heavy-censored		
	C-index	MSE	MAE	C-index	MSE	MAE	C-index	MSE	MAE
IC_tree	0.773	9.827	2.476	0.762	Inf	Inf	0.743	Inf	Inf
ctree	0.796	8.244	2.279	0.783	Inf	Inf	0.759	Inf	Inf
Middle									
regression	0.804	7.737	2.203	0.802	8.516	2.297	0.791	10.293	2.475
bagging	0.896	2.908	1.318	0.891	3.715	1.443	0.883	5.845	1.713
randomforest	0.804	2.931	1.317	0.802	3.614	1.411	0.791	5.76	1.684
boosting	0.893	2.816	1.312	0.885	3.83	1.488	0.875	5.83	1.739
xgboost	0.891	2.85	1.321	0.883	3.949	1.508	0.872	6.042	1.772
BART	0.921	1.54	0.97	0.917	2.287	1.119	0.91	4.25	1.398
ctree	0.813	7.367	2.146	0.807	8.307	2.258	0.797	10.111	2.453
rpart	0.785	9.003	2.368	0.775	Inf	Inf	0.755	Inf	Inf
XGB-GAM									
regression	0.805	7.736	2.203	0.801	8.508	2.295	0.791	10.287	2.476
bagging	0.898	2.846	1.302	0.892	3.685	1.434	0.884	5.826	1.707
randomforest	0.805	2.903	1.309	0.801	3.617	1.41	0.791	5.778	1.687
boosting	0.9	2.536	1.236	0.89	3.61	1.434	0.879	5.675	1.701
xgboost	0.897	2.627	1.265	0.887	3.758	1.458	0.875	5.909	1.739
BART	0.922	1.499	0.958	0.918	2.287	1.118	0.911	4.258	1.397
ctree	0.812	7.369	2.144	0.808	8.207	2.241	0.798	10.035	2.443
rpart	0.798	8.325	2.273	0.789	Inf	Inf	0.769	Inf	Inf

Table 5.5: Linear model for Large Interval with n=200.

Method	no-censored			light-censored			heavy-censored		
	C-index	MSE	MAE	C-index	MSE	MAE	C-index	MSE	MAE
IC_tree	0.769	74.152	6.738	0.764	Inf	Inf	0.75	Inf	Inf
ctree	0.805	56.877	6.001	0.789	Inf	Inf	0.77	Inf	Inf
Middle									
regression	0.813	51.542	5.698	0.808	66.428	6.405	0.804	70.31	6.546
bagging	0.912	17.539	3.168	0.907	24.62	3.704	0.903	31.291	4.107
randomforest	0.813	18.268	3.146	0.808	21.569	3.44	0.804	28.689	3.889
boosting	0.921	14.488	2.762	0.913	26.401	3.755	0.907	32.799	4.163
xgboost	0.923	12.4	2.61	0.914	23.697	3.565	0.907	30.736	4.032
BART	0.955	5.178	1.584	0.948	15.121	2.677	0.944	20.967	3.153
ctree	0.812	53.271	5.801	0.807	67.023	6.423	0.801	71.725	6.636
rpart	0.799	59.656	6.115	0.788	Inf	Inf	0.771	Inf	Inf
XGB-GAM									
regression	0.814	51.337	5.687	0.809	65.696	6.378	0.806	69.716	6.518
bagging	0.914	17.285	3.14	0.908	24.472	3.687	0.903	31.243	4.098
randomforest	0.814	18.149	3.13	0.809	21.507	3.433	0.806	28.733	3.891
boosting	0.926	13.263	2.591	0.917	25.528	3.659	0.91	31.991	4.073
xgboost	0.928	11.403	2.471	0.917	23.355	3.493	0.909	30.176	3.965
BART	0.957	5.03	1.513	0.949	15.301	2.671	0.945	21.231	3.15
ctree	0.812	52.877	5.777	0.808	66.915	6.427	0.803	71.216	6.603
rpart	0.804	57.343	5.983	0.795	Inf	Inf	0.778	Inf	Inf

Table 5.6: Linear model for Large Interval with n=300.

Method	no-censored			light-censored			heavy-censored		
	C-index	MSE	MAE	C-index	MSE	MAE	C-index	MSE	MAE
IC_tree	0.794	61.246	6.098	0.787	Inf	Inf	0.771	Inf	Inf
ctree	0.825	46.354	5.389	0.81	Inf	Inf	0.79	Inf	Inf
Middle									
regression	0.822	46.906	5.431	0.817	57.741	5.981	0.813	64.679	6.269
bagging	0.921	13.847	2.812	0.916	20.051	3.306	0.91	29.177	3.847
randomforest	0.822	14.18	2.758	0.817	17.777	3.054	0.813	27.121	3.628
boosting	0.935	9.853	2.256	0.927	18.9	3.115	0.918	28.4	3.724
xgboost	0.932	9.57	2.298	0.922	19.015	3.149	0.913	28.418	3.742
BART	0.96	3.684	1.364	0.955	11.383	2.272	0.948	19.976	2.914
ctree	0.835	42.908	5.176	0.83	53.241	5.71	0.824	61.148	6.068
rpart	0.811	53.434	5.765	0.802	Inf	Inf	0.786	Inf	Inf
XGB-GAM									
regression	0.822	47.009	5.435	0.817	57.746	5.987	0.813	64.986	6.281
bagging	0.923	13.541	2.775	0.917	19.791	3.273	0.911	29.107	3.832
randomforest	0.822	14.06	2.738	0.817	17.618	3.031	0.813	27.156	3.627
boosting	0.941	8.695	2.062	0.932	18.091	2.991	0.921	27.74	3.637
xgboost	0.938	8.633	2.138	0.927	18.19	3.021	0.916	27.949	3.665
BART	0.963	3.472	1.276	0.956	11.565	2.256	0.948	20.273	2.924
ctree	0.836	42.498	5.15	0.831	52.687	5.681	0.825	61.084	6.058
rpart	0.817	51.066	5.635	0.81	Inf	Inf	0.795	Inf	Inf

Table 5.7: Boxcox model for Small Interval with n=200.

Method	no-censored			light-censored			heavy-censored		
	C-index	MSE	MAE	C-index	MSE	MAE	C-index	MSE	MAE
IC_tree	0.755	38.245	4.67	0.752	Inf	Inf	0.734	Inf	Inf
ctree	0.816	24.9	3.856	0.805	Inf	Inf	0.771	Inf	Inf
Middle									
regression	0.816	24.234	3.829	0.818	26.168	3.855	0.814	32.505	4.164
bagging	0.875	13.397	2.802	0.875	18.224	3.089	0.872	25.849	3.558
randomforest	0.816	14.471	2.861	0.818	20.395	3.217	0.814	28.268	3.687
boosting	0.855	18.878	3.293	0.859	20.147	3.262	0.854	26.566	3.614
xgboost	0.866	14.42	2.918	0.866	18.104	3.133	0.862	25.301	3.555
BART	0.886	11.048	2.526	0.888	14.602	2.74	0.885	21.841	3.215
ctree	0.817	24.303	3.833	0.818	26.255	3.869	0.814	32.437	4.185
rpart	0.812	26.23	3.941	0.806	Inf	Inf	0.778	Inf	Inf
XGB-GAM									
regression	0.816	24.242	3.83	0.818	26.157	3.852	0.814	32.503	4.166
bagging	0.875	13.381	2.8	0.875	18.248	3.091	0.871	25.834	3.557
randomforest	0.816	14.463	2.861	0.818	20.406	3.218	0.814	28.275	3.686
boosting	0.855	18.88	3.288	0.859	20.071	3.25	0.853	26.563	3.616
xgboost	0.866	14.38	2.916	0.867	18.058	3.126	0.862	25.315	3.559
BART	0.886	11.049	2.527	0.888	14.603	2.741	0.885	21.845	3.215
ctree	0.817	24.3	3.831	0.818	26.249	3.87	0.814	32.42	4.184
rpart	0.812	26.191	3.936	0.806	Inf	Inf	0.779	Inf	Inf

Table 5.8: Boxcox model for Small Interval with n=300.

Method	no-censored			light-censored			heavy-censored		
	C-index	MSE	MAE	C-index	MSE	MAE	C-index	MSE	MAE
IC_tree	0.778	33.982	4.356	0.779	Inf	Inf	0.754	Inf	Inf
ctree	0.832	22.073	3.611	0.819	Inf	Inf	0.784	Inf	Inf
Middle									
regression	0.821	23.16	3.751	0.822	27.094	3.888	0.82	34.37	4.224
bagging	0.881	12.376	2.696	0.879	18.914	3.099	0.877	27.585	3.617
randomforest	0.821	13.117	2.731	0.822	20.784	3.188	0.82	29.802	3.718
boosting	0.866	16.313	3.058	0.867	19.845	3.189	0.861	27.961	3.663
xgboost	0.871	13.723	2.856	0.87	19.07	3.163	0.865	27.575	3.662
BART	0.891	10.207	2.436	0.891	15.459	2.771	0.888	24.056	3.33
ctree	0.836	21.312	3.545	0.833	25.189	3.732	0.832	32.522	4.104
rpart	0.821	25.191	3.834	0.814	Inf	Inf	0.788	Inf	Inf
XGB-GAM									
regression	0.82	23.158	3.75	0.822	27.093	3.887	0.819	34.383	4.225
bagging	0.881	12.382	2.696	0.879	18.91	3.099	0.877	27.572	3.616
randomforest	0.82	13.137	2.732	0.822	20.783	3.188	0.819	29.785	3.717
boosting	0.866	16.231	3.051	0.867	19.822	3.187	0.861	27.904	3.657
xgboost	0.872	13.673	2.85	0.87	19.001	3.155	0.865	27.561	3.66
BART	0.891	10.215	2.436	0.891	15.457	2.77	0.888	24.055	3.33
ctree	0.836	21.289	3.545	0.833	25.207	3.733	0.831	32.521	4.104
rpart	0.821	25.148	3.834	0.814	Inf	Inf	0.789	Inf	Inf

Table 5.9: Boxcox model for Medium Interval with n=200.

Method	no-censored			light-censored			heavy-censored		
	C-index	MSE	MAE	C-index	MSE	MAE	C-index	MSE	MAE
IC_tree	0.765	220.833	11.538	0.758	Inf	Inf	0.734	Inf	Inf
ctree	0.788	189.352	10.684	0.78	Inf	Inf	0.752	Inf	Inf
Middle									
regression	0.8	167.3	10.07	0.799	166.984	10.093	0.795	185.27	10.552
bagging	0.896	66.266	6.046	0.895	70.502	6.204	0.89	88.051	6.848
randomforest	0.8	69.152	6.176	0.799	73.048	6.311	0.795	89.508	6.895
boosting	0.894	59.519	5.841	0.895	60.869	5.908	0.889	80.744	6.7
xgboost	0.894	58.971	5.821	0.892	64.28	6.058	0.887	84.075	6.825
BART	0.921	33.253	4.385	0.922	35.692	4.496	0.919	50.916	5.224
ctree	0.793	178.239	10.398	0.792	177.404	10.372	0.789	190.101	10.741
rpart	0.786	189.377	10.682	0.781	Inf	Inf	0.757	Inf	Inf
XGB-GAM									
regression	0.799	167.617	10.074	0.8	166.85	10.101	0.795	185.049	10.546
bagging	0.896	66.108	6.036	0.895	70.49	6.202	0.891	87.908	6.837
randomforest	0.799	69.104	6.169	0.8	72.912	6.301	0.795	89.468	6.888
boosting	0.894	59.385	5.836	0.895	60.876	5.901	0.889	80.392	6.674
xgboost	0.894	59.143	5.828	0.892	63.933	6.043	0.887	84.192	6.832
BART	0.921	33.207	4.381	0.922	35.605	4.493	0.919	50.802	5.216
ctree	0.793	178.451	10.404	0.792	177.315	10.37	0.79	190.071	10.728
rpart	0.785	189.356	10.692	0.782	Inf	Inf	0.758	Inf	Inf

Table 5.10: Boxcox model for Medium Interval with n=300.

Method	no-censored			light-censored			heavy-censored		
	C-index	MSE	MAE	C-index	MSE	MAE	C-index	MSE	MAE
IC_tree	0.784	188.907	10.645	0.779	Inf	Inf	0.754	Inf	Inf
ctree	0.813	153.719	9.565	0.806	Inf	Inf	0.774	Inf	Inf
Middle									
regression	0.806	152.562	9.657	0.808	155.421	9.71	0.806	175.146	10.198
bagging	0.904	53.461	5.447	0.902	62.95	5.779	0.9	84.101	6.538
randomforest	0.806	55.662	5.543	0.808	65.219	5.859	0.806	86.143	6.596
boosting	0.905	46.205	5.17	0.904	52.652	5.421	0.899	75.609	6.329
xgboost	0.9	49.924	5.408	0.899	57.989	5.701	0.895	80.982	6.574
BART	0.925	28.546	4.107	0.925	33.5	4.329	0.923	52.169	5.153
ctree	0.818	144.924	9.304	0.816	148.458	9.432	0.815	164.679	9.876
rpart	0.799	167.46	10.011	0.796	Inf	Inf	0.776	Inf	Inf
XGB-GAM									
regression	0.806	152.483	9.658	0.808	156.121	9.721	0.806	174.847	10.182
bagging	0.904	53.419	5.442	0.902	62.853	5.772	0.9	84.044	6.533
randomforest	0.806	55.573	5.538	0.808	65.115	5.855	0.806	86.146	6.593
boosting	0.905	46.008	5.159	0.904	52.756	5.424	0.899	75.27	6.31
xgboost	0.901	49.674	5.39	0.899	57.651	5.682	0.895	81.022	6.576
BART	0.925	28.591	4.109	0.925	33.452	4.324	0.923	52.2	5.156
ctree	0.818	144.849	9.308	0.816	148.595	9.429	0.815	164.542	9.874
rpart	0.8	166.584	9.988	0.798	Inf	Inf	0.777	Inf	Inf

Table 5.11: Boxcox model for Large Interval with n=200.

Method	no-censored			light-censored			heavy-censored		
	C-index	MSE	MAE	C-index	MSE	MAE	C-index	MSE	MAE
IC_tree	0.763	221.704	11.548	0.757	Inf	Inf	0.735	Inf	Inf
ctree	0.787	192.118	10.725	0.782	Inf	Inf	0.753	Inf	Inf
Middle									
regression	0.799	166.313	10.066	0.799	172.851	10.28	0.795	190.632	10.711
bagging	0.895	66.044	6.027	0.894	68.175	6.166	0.889	95.098	7.094
randomforest	0.799	69.636	6.183	0.799	69.329	6.217	0.795	96.07	7.126
boosting	0.892	60.49	5.893	0.891	67.453	6.254	0.884	91.766	7.115
xgboost	0.892	59.814	5.871	0.89	66.974	6.224	0.882	94.396	7.218
BART	0.92	33.444	4.392	0.921	37.04	4.642	0.917	58.041	5.557
ctree	0.792	178.098	10.364	0.791	183.695	10.56	0.787	198.941	10.961
rpart	0.786	187.96	10.626	0.779	Inf	Inf	0.758	Inf	Inf
XGB-GAM									
regression	0.798	166.772	10.091	0.798	174.277	10.325	0.795	189.74	10.689
bagging	0.896	65.971	6.02	0.894	68.221	6.16	0.889	95.11	7.088
randomforest	0.798	69.528	6.174	0.798	69.334	6.215	0.795	96.176	7.124
boosting	0.893	59.98	5.875	0.892	66.358	6.192	0.884	91.052	7.092
xgboost	0.893	59.26	5.842	0.89	66.922	6.22	0.883	94.205	7.183
BART	0.921	33.312	4.38	0.921	37.009	4.642	0.917	58.11	5.557
ctree	0.792	177.959	10.374	0.791	183.156	10.549	0.789	197.057	10.91
rpart	0.787	186.899	10.611	0.781	Inf	Inf	0.76	Inf	Inf

Table 5.12: Boxcox model for Large Interval with n=300.

Method	no-censored			light-censored			heavy-censored		
	C-index	MSE	MAE	C-index	MSE	MAE	C-index	MSE	MAE
IC_tree	0.785	191.445	10.712	0.78	Inf	Inf	0.757	Inf	Inf
ctree	0.812	156.065	9.635	0.808	Inf	Inf	0.775	Inf	Inf
Middle									
regression	0.808	151.716	9.612	0.808	158.063	9.822	0.804	181.451	10.373
bagging	0.904	54.727	5.482	0.903	58.767	5.662	0.897	89.18	6.728
randomforest	0.808	57.358	5.595	0.808	59.959	5.698	0.804	89.965	6.733
boosting	0.904	48.257	5.281	0.903	54.592	5.597	0.894	83.957	6.677
xgboost	0.899	51.441	5.476	0.898	58.604	5.785	0.89	90.196	6.949
BART	0.925	29.369	4.133	0.925	33.244	4.373	0.921	56.465	5.362
ctree	0.818	145.632	9.333	0.818	149.912	9.513	0.812	171.842	10.062
rpart	0.8	170.368	10.084	0.798	Inf	Inf	0.773	Inf	Inf
XGB-GAM									
regression	0.808	151.558	9.609	0.809	158.25	9.829	0.805	181.018	10.356
bagging	0.904	54.691	5.474	0.904	58.571	5.647	0.898	88.989	6.72
randomforest	0.808	57.229	5.583	0.809	59.84	5.69	0.805	89.792	6.726
boosting	0.905	47.575	5.236	0.903	54.271	5.566	0.894	83.709	6.65
xgboost	0.9	50.855	5.441	0.899	58.298	5.765	0.89	89.527	6.907
BART	0.925	29.109	4.111	0.925	33.244	4.368	0.921	56.482	5.363
ctree	0.818	145.502	9.334	0.818	149.174	9.487	0.813	171.185	10.044
rpart	0.801	169.215	10.043	0.8	Inf	Inf	0.777	Inf	Inf

Table 5.13: PH model for Small Interval with n=200.

Method	no-censored			light-censored			heavy-censored		
	C-index	MSE	MAE	C-index	MSE	MAE	C-index	MSE	MAE
IC_tree	0.753	7.507	2.136	0.75	Inf	Inf	0.726	Inf	Inf
ctree	0.781	6.276	1.957	0.777	Inf	Inf	0.744	Inf	Inf
Middle									
regression	0.79	5.734	1.875	0.787	6.03	1.913	0.774	7.229	2.065
bagging	0.863	3.081	1.338	0.859	3.725	1.456	0.847	5.342	1.709
randomforest	0.79	3.12	1.344	0.787	3.828	1.472	0.774	5.464	1.725
boosting	0.845	3.696	1.489	0.844	3.928	1.523	0.829	5.34	1.739
xgboost	0.853	3.285	1.393	0.85	3.756	1.477	0.835	5.322	1.721
BART	0.882	2.229	1.137	0.88	2.597	1.226	0.873	4.025	1.496
ctree	0.783	6.03	1.923	0.781	6.299	1.958	0.769	7.472	2.11
rpart	0.78	6.303	1.961	0.777	Inf	Inf	0.752	Inf	Inf
XGB-GAM									
regression	0.79	5.752	1.877	0.788	6.026	1.912	0.774	7.226	2.066
bagging	0.863	3.078	1.337	0.859	3.726	1.456	0.848	5.338	1.708
randomforest	0.79	3.119	1.344	0.788	3.824	1.471	0.774	5.458	1.724
boosting	0.846	3.667	1.482	0.845	3.919	1.52	0.829	5.325	1.737
xgboost	0.854	3.266	1.388	0.851	3.73	1.47	0.835	5.326	1.72
BART	0.882	2.228	1.137	0.881	2.595	1.225	0.873	4.024	1.496
ctree	0.783	6.058	1.927	0.78	6.317	1.959	0.769	7.481	2.112
rpart	0.781	6.257	1.954	0.778	Inf	Inf	0.754	Inf	Inf

Table 5.14: PH model for Small Interval with n=300.

Method	no-censored			light-censored			heavy-censored		
	C-index	MSE	MAE	C-index	MSE	MAE	C-index	MSE	MAE
IC_tree	0.771	6.648	2.001	0.766	Inf	Inf	0.738	Inf	Inf
ctree	0.801	5.418	1.808	0.793	Inf	Inf	0.756	Inf	Inf
Middle									
regression	0.794	5.508	1.841	0.793	5.75	1.862	0.777	7.016	2.023
bagging	0.867	2.813	1.286	0.863	3.499	1.407	0.851	5.225	1.678
randomforest	0.794	2.808	1.282	0.793	3.557	1.412	0.777	5.311	1.686
boosting	0.856	3.173	1.383	0.853	3.569	1.446	0.834	5.176	1.7
xgboost	0.857	3.063	1.353	0.854	3.589	1.439	0.836	5.28	1.708
BART	0.884	2.062	1.103	0.882	2.512	1.204	0.873	4.074	1.497
ctree	0.802	5.3	1.796	0.798	5.591	1.83	0.783	6.919	2.013
rpart	0.79	5.837	1.885	0.788	Inf	Inf	0.761	Inf	Inf
XGB-GAM									
regression	0.794	5.53	1.844	0.793	5.757	1.864	0.777	7.019	2.025
bagging	0.867	2.811	1.286	0.863	3.501	1.408	0.851	5.224	1.678
randomforest	0.794	2.806	1.282	0.793	3.557	1.412	0.777	5.312	1.686
boosting	0.856	3.163	1.381	0.853	3.573	1.447	0.834	5.172	1.7
xgboost	0.858	3.046	1.348	0.854	3.59	1.439	0.836	5.289	1.709
BART	0.884	2.063	1.104	0.882	2.516	1.205	0.874	4.075	1.497
ctree	0.802	5.309	1.797	0.798	5.595	1.831	0.783	6.921	2.014
rpart	0.79	5.828	1.882	0.789	Inf	Inf	0.763	Inf	Inf

Table 5.15: PH model Medium Interval with n=200

Method	no-censored			light-censored			heavy-censored		
	C-index	MSE	MAE	C-index	MSE	MAE	C-index	MSE	MAE
IC_tree	0.749	12.991	2.824	0.737	Inf	Inf	0.71	Inf	Inf
ctree	0.77	11.48	2.658	0.748	Inf	Inf	0.72	Inf	Inf
Middle									
regression	0.786	10.225	2.498	0.779	11.658	2.647	0.764	14.065	2.854
bagging	0.875	4.714	1.645	0.865	6.268	1.834	0.852	9.648	2.195
randomforest	0.786	4.757	1.642	0.779	6.167	1.802	0.764	9.603	2.174
boosting	0.864	5.153	1.735	0.854	6.673	1.933	0.838	9.606	2.236
xgboost	0.867	4.694	1.655	0.855	6.508	1.896	0.839	9.736	2.231
bart	0.902	2.703	1.238	0.898	4.067	1.433	0.889	7.233	1.829
ctree	0.778	10.862	2.58	0.768	12.479	2.744	0.751	14.994	2.956
rpart	0.764	11.756	2.683	0.75	Inf	Inf	0.73	Inf	Inf
XGB-GAM									
regression	0.786	10.239	2.499	0.777	11.685	2.652	0.763	14.224	2.871
bagging	0.876	4.679	1.638	0.866	6.218	1.825	0.853	9.638	2.191
randomforest	0.786	4.757	1.643	0.777	6.143	1.799	0.763	9.621	2.177
boosting	0.867	4.9	1.688	0.857	6.512	1.899	0.842	9.514	2.219
xgboost	0.872	4.498	1.617	0.858	6.375	1.867	0.842	9.724	2.223
bart	0.903	2.68	1.24	0.899	4.059	1.434	0.89	7.25	1.84
ctree	0.779	10.782	2.567	0.768	12.489	2.74	0.753	14.99	2.958
rpart	0.773	11.399	2.635	0.76	Inf	Inf	0.738	Inf	Inf

Table 5.16: PH model Medium Interval with n=300

Method	no-censored			light-censored			heavy-censored		
	C-index	MSE	MAE	C-index	MSE	MAE	C-index	MSE	MAE
IC_tree	0.77	11.266	2.623	0.754	Inf	Inf	0.727	Inf	Inf
ctree	0.786	10.065	2.485	0.768	Inf	Inf	0.736	Inf	Inf
Middle									
regression	0.792	9.574	2.428	0.786	10.519	2.52	0.772	13.214	2.75
bagging	0.88	4.105	1.543	0.872	5.442	1.709	0.86	9.059	2.106
randomforest	0.792	4.07	1.523	0.786	5.322	1.67	0.772	9.012	2.08
boosting	0.875	4.158	1.569	0.865	5.587	1.767	0.847	8.984	2.135
xgboost	0.874	4.142	1.565	0.863	5.727	1.783	0.845	9.284	2.168
bart	0.907	2.362	1.168	0.902	3.48	1.333	0.892	6.908	1.764
ctree	0.799	9.299	2.377	0.789	10.518	2.509	0.775	13.314	2.763
rpart	0.773	10.952	2.585	0.762	Inf	Inf	0.74	Inf	Inf
XGB-GAM									
regression	0.792	9.538	2.421	0.786	10.45	2.512	0.772	13.253	2.755
bagging	0.882	4.059	1.533	0.874	5.394	1.701	0.862	9.055	2.108
randomforest	0.792	4.057	1.519	0.786	5.296	1.667	0.772	9.03	2.085
boosting	0.88	3.911	1.517	0.868	5.455	1.741	0.85	8.938	2.129
xgboost	0.879	3.886	1.513	0.865	5.64	1.761	0.848	9.232	2.157
bart	0.908	2.339	1.169	0.903	3.474	1.343	0.893	6.94	1.779
ctree	0.799	9.294	2.377	0.791	10.315	2.486	0.775	13.356	2.766
rpart	0.785	10.246	2.495	0.776	Inf	Inf	0.752	Inf	Inf

Table 5.17: PH model for Large Interval with n=200.

Method	no-censored			light-censored			heavy-censored		
	C-index	MSE	MAE	C-index	MSE	MAE	C-index	MSE	MAE
IC_tree	0.768	76.643	6.81	0.764	Inf	Inf	0.733	Inf	Inf
ctree	0.8	60.557	6.092	0.786	Inf	Inf	0.746	Inf	Inf
Middle									
regression	0.808	56.772	5.911	0.806	71.232	6.532	0.79	83.386	6.968
bagging	0.904	20.916	3.409	0.905	28.247	3.858	0.888	49.489	4.927
randomforest	0.808	21.31	3.359	0.806	25.292	3.598	0.79	48.557	4.813
boosting	0.909	18.614	3.131	0.911	30.625	3.938	0.889	47.92	4.811
xgboost	0.916	15.126	2.857	0.913	26.585	3.648	0.89	47.09	4.737
BART	0.948	6.859	1.812	0.954	17.096	2.648	0.932	35.26	3.846
ctree	0.809	57.224	5.912	0.806	71.664	6.531	0.79	84.445	7.047
rpart	0.793	64.573	6.282	0.784	Inf	Inf	0.751	Inf	Inf
XGB-GAM									
regression	0.809	56.328	5.884	0.805	71.665	6.545	0.792	83.154	6.965
bagging	0.905	20.689	3.383	0.907	28.115	3.839	0.889	49.245	4.9
randomforest	0.809	21.276	3.351	0.805	25.347	3.597	0.792	48.218	4.777
boosting	0.914	17.54	2.993	0.915	29.562	3.816	0.892	47.026	4.739
xgboost	0.921	14.15	2.737	0.918	25.828	3.536	0.893	46.515	4.667
BART	0.949	6.753	1.779	0.955	17.65	2.645	0.932	35.298	3.819
ctree	0.809	56.958	5.908	0.807	71.554	6.518	0.792	84.526	7.046
rpart	0.798	62.716	6.196	0.792	Inf	Inf	0.758	Inf	Inf

0

Table 5.18: PH model for Large Interval with n=300.

Method	no-censored			light-censored			heavy-censored		
	C-index	MSE	MAE	C-index	MSE	MAE	C-index	MSE	MAE
IC_tree	0.796	62.708	6.126	0.785	Inf	Inf	0.75	Inf	Inf
ctree	0.821	51.127	5.581	0.808	Inf	Inf	0.763	Inf	Inf
Middle									
regression	0.816	52.137	5.671	0.814	63.424	6.192	0.799	76.57	6.664
bagging	0.915	16.928	3.035	0.915	23.805	3.499	0.896	44.635	4.591
randomforest	0.816	17.061	2.967	0.814	21.144	3.24	0.799	43.614	4.468
boosting	0.927	12.625	2.538	0.926	23.646	3.383	0.9	41.982	4.391
xgboost	0.925	12.251	2.562	0.922	22.83	3.332	0.896	42.932	4.466
BART	0.954	5.189	1.575	0.958	15.029	2.427	0.935	32.32	3.581
ctree	0.829	47.643	5.396	0.826	59.684	5.959	0.81	73.363	6.506
rpart	0.806	58.278	5.956	0.798	Inf	Inf	0.763	Inf	Inf
XGB-GAM									
regression	0.816	52.115	5.673	0.814	63.72	6.205	0.8	76.08	6.643
bagging	0.916	16.77	3.005	0.917	23.72	3.484	0.897	44.429	4.566
randomforest	0.816	17.051	2.955	0.814	21.206	3.243	0.8	43.464	4.443
boosting	0.931	11.701	2.399	0.931	22.699	3.237	0.902	41.33	4.328
xgboost	0.93	11.34	2.426	0.927	22.182	3.219	0.899	42.406	4.396
BART	0.956	5.108	1.536	0.959	15.682	2.426	0.935	32.464	3.575
ctree	0.83	47.356	5.367	0.827	59.61	5.953	0.811	72.953	6.487
rpart	0.812	56.444	5.851	0.806	Inf	Inf	0.77	Inf	Inf

Table 5.19: AFT model for Small Interval with n=200.

Method	no-censored			light-censored			heavy-censored		
	C-index	MSE	MAE	C-index	MSE	MAE	C-index	MSE	MAE
IC_tree	0.724	23.214	3.407	0.722	Inf	Inf	0.703	Inf	Inf
ctree	0.76	19.355	3.082	0.758	Inf	Inf	0.741	Inf	Inf
Middle									
regression	0.764	20.069	3.178	0.767	18.884	3.023	0.758	26.265	3.423
bagging	0.788	17.454	2.972	0.786	17.995	2.912	0.78	25.663	3.349
randomforest	0.764	16.686	2.913	0.767	18.074	2.906	0.758	26.013	3.369
boosting	0.76	21.911	3.461	0.767	19.701	3.153	0.76	26.268	3.454
xgboost	0.769	21.026	3.259	0.769	19.598	3.088	0.762	26.463	3.452
BART	0.801	15.868	2.851	0.804	16.94	2.801	0.802	24.667	3.268
ctree	0.761	19.112	3.119	0.76	19.072	3.02	0.753	26.223	3.412
rpart	0.771	19.284	3.101	0.768	Inf	Inf	0.755	Inf	Inf
XGB-GAM									
regression	0.764	20.04	3.175	0.766	18.885	3.022	0.758	26.256	3.421
bagging	0.789	17.441	2.971	0.786	17.982	2.911	0.78	25.669	3.349
randomforest	0.764	16.667	2.912	0.766	18.071	2.907	0.758	26.009	3.368
boosting	0.761	21.771	3.445	0.768	19.602	3.138	0.761	26.235	3.446
xgboost	0.77	21.151	3.26	0.769	19.526	3.08	0.762	26.449	3.448
BART	0.801	15.876	2.853	0.804	16.943	2.802	0.802	24.689	3.27
ctree	0.761	19.11	3.121	0.76	19.038	3.016	0.753	26.22	3.411
rpart	0.771	19.171	3.088	0.769	Inf	Inf	0.756	Inf	Inf

Table 5.20: AFT model for Small Interval with n=300.

Method	no-censored			light-censored			heavy-censored		
	C-index	MSE	MAE	C-index	MSE	MAE	C-index	MSE	MAE
IC_tree	0.741	21.37	3.252	0.735	Inf	Inf	0.72	Inf	Inf
ctree	0.768	18.29	2.999	0.766	Inf	Inf	0.749	Inf	Inf
Middle									
regression	0.765	18.605	3.063	0.768	18.584	2.989	0.759	25.857	3.383
bagging	0.791	16.626	2.903	0.788	17.655	2.895	0.784	25.219	3.315
randomforest	0.765	15.998	2.847	0.768	17.668	2.883	0.759	25.398	3.321
boosting	0.763	20.619	3.342	0.771	19.095	3.101	0.764	25.744	3.42
xgboost	0.77	20.148	3.205	0.771	19.145	3.068	0.764	25.909	3.415
BART	0.802	15.278	2.796	0.806	16.653	2.785	0.804	24.25	3.233
ctree	0.769	18.071	3.032	0.766	18.677	2.992	0.76	25.7	3.37
rpart	0.773	18.256	3.002	0.772	Inf	Inf	0.756	Inf	Inf
XGB-GAM									
regression	0.765	18.62	3.062	0.768	18.557	2.988	0.759	25.844	3.381
bagging	0.791	16.623	2.903	0.788	17.642	2.894	0.784	25.216	3.315
randomforest	0.765	15.997	2.848	0.768	17.663	2.882	0.759	25.398	3.322
boosting	0.764	20.571	3.342	0.771	19.076	3.1	0.764	25.722	3.415
xgboost	0.771	20.114	3.199	0.771	19.065	3.059	0.765	25.863	3.41
BART	0.802	15.273	2.796	0.806	16.653	2.785	0.804	24.263	3.235
ctree	0.769	18.063	3.03	0.766	18.65	2.991	0.761	25.705	3.37
rpart	0.773	18.275	3	0.773	Inf	Inf	0.756	Inf	Inf

Table 5.21: AFT model for Medium Interval with n=200.

Method	no-censored			light-censored			heavy-censored		
	C-index	MSE	MAE	C-index	MSE	MAE	C-index	MSE	MAE
IC_tree	0.754	77.898	6.049	0.742	Inf	Inf	0.721	Inf	Inf
ctree	0.782	66.188	5.549	0.78	Inf	Inf	0.761	Inf	Inf
Middle									
regression	0.779	65.476	5.568	0.787	61.579	5.274	0.78	86.233	5.978
bagging	0.821	53.546	4.999	0.819	56.254	4.933	0.812	82.951	5.771
randomforest	0.779	51.095	4.89	0.787	56.541	4.92	0.78	84.113	5.804
boosting	0.792	66.094	5.844	0.801	60.634	5.347	0.794	84.338	5.977
xgboost	0.803	65.43	5.511	0.802	61.423	5.256	0.793	85.332	5.967
BART	0.832	48.828	4.834	0.837	52.678	4.738	0.836	78.999	5.586
ctree	0.785	64.857	5.549	0.784	62.829	5.32	0.777	86.47	5.996
rpart	0.793	62.892	5.407	0.788	Inf	Inf	0.77	Inf	Inf
XGB-GAM									
regression	0.779	64.601	5.535	0.786	61.579	5.272	0.779	86.132	5.973
bagging	0.822	53.294	4.987	0.82	56.164	4.923	0.812	82.959	5.768
randomforest	0.779	50.927	4.884	0.786	56.495	4.914	0.779	84.151	5.805
boosting	0.795	64.527	5.758	0.803	59.992	5.294	0.796	84.074	5.954
xgboost	0.806	64.152	5.445	0.804	60.895	5.203	0.794	85.203	5.941
BART	0.831	48.862	4.839	0.836	52.723	4.739	0.835	79.1	5.592
ctree	0.787	64.588	5.543	0.786	62.62	5.304	0.778	86.465	5.995
rpart	0.795	62.361	5.358	0.794	Inf	Inf	0.774	Inf	Inf

Table 5.22: AFT model for Medium Interval with n=300.

Method	no-censored			light-censored			heavy-censored		
	C-index	MSE	MAE	C-index	MSE	MAE	C-index	MSE	MAE
IC_tree	0.768	70.798	5.75	0.763	Inf	Inf	0.737	Inf	Inf
ctree	0.791	60.768	5.33	0.789	Inf	Inf	0.766	Inf	Inf
Middle									
regression	0.78	60.594	5.402	0.788	61.244	5.232	0.777	85.156	5.894
bagging	0.824	49.895	4.856	0.823	55.732	4.876	0.812	81.365	5.658
randomforest	0.78	47.759	4.762	0.788	55.909	4.861	0.777	81.965	5.663
boosting	0.795	61.591	5.644	0.804	59.735	5.28	0.796	82.571	5.854
xgboost	0.805	60.884	5.369	0.805	60.402	5.202	0.794	83.615	5.848
BART	0.832	45.754	4.726	0.838	52.381	4.708	0.834	77.475	5.483
ctree	0.794	59.593	5.336	0.793	61.726	5.228	0.783	84.567	5.87
rpart	0.793	60.081	5.295	0.792	Inf	Inf	0.77	Inf	Inf
XGB-GAM									
regression	0.78	60.254	5.395	0.789	60.914	5.216	0.777	85.114	5.894
bagging	0.825	49.59	4.845	0.823	55.625	4.868	0.813	81.346	5.654
randomforest	0.78	47.589	4.757	0.789	55.793	4.854	0.777	81.98	5.663
boosting	0.797	60.659	5.586	0.806	59.038	5.219	0.798	82.214	5.825
xgboost	0.808	60.106	5.321	0.808	59.673	5.141	0.796	83.262	5.817
BART	0.832	45.816	4.733	0.837	52.385	4.71	0.834	77.6	5.489
ctree	0.796	59.429	5.328	0.796	61.408	5.206	0.784	84.602	5.869
rpart	0.796	59.864	5.25	0.797	Inf	Inf	0.776	Inf	Inf

Table 5.23: AFT model for Large Interval with n=200.

Method	no-censored			light-censored			heavy-censored		
	C-index	MSE	MAE	C-index	MSE	MAE	C-index	MSE	MAE
IC_tree	0.762	266.423	10.567	0.748	Inf	Inf	0.715	Inf	Inf
ctree	0.798	197.324	8.989	0.798	Inf	Inf	0.784	Inf	Inf
Middle									
regression	0.784	192.308	8.898	0.785	190.478	8.619	0.779	237.619	9.341
bagging	0.841	157.408	7.977	0.841	174.178	7.99	0.833	226.49	8.874
randomforest	0.784	150.565	7.871	0.785	177.924	8.056	0.779	231.556	8.983
boosting	0.811	194.206	9.706	0.821	184.382	8.804	0.818	228.392	9.309
xgboost	0.826	190.918	8.818	0.825	186.26	8.478	0.817	233.333	9.221
BART	0.85	145.35	8.011	0.855	167.419	7.886	0.852	217.43	8.691
ctree	0.804	192.709	8.915	0.805	193.417	8.612	0.797	236.032	9.235
rpart	0.803	189.813	8.827	0.798	Inf	Inf	0.783	Inf	Inf
XGB-GAM									
regression	0.784	190.759	8.889	0.785	190.294	8.615	0.779	237.755	9.355
bagging	0.842	156.715	7.972	0.842	174.224	7.988	0.833	226.675	8.883
randomforest	0.784	150.359	7.882	0.785	178.237	8.073	0.779	231.77	8.998
boosting	0.814	189.262	9.534	0.823	183.241	8.727	0.819	228.024	9.262
xgboost	0.829	188.549	8.733	0.827	184.949	8.407	0.818	232.962	9.168
BART	0.849	145.576	8.015	0.855	167.64	7.901	0.852	217.68	8.709
ctree	0.807	191.509	8.896	0.807	193.437	8.611	0.799	236.347	9.244
rpart	0.811	189.678	8.684	0.81	Inf	Inf	0.794	Inf	Inf

Table 5.24: AFT model for Large Interval with n=300.

Method	no-censored			light-censored			heavy-censored		
	C-index	MSE	MAE	C-index	MSE	MAE	C-index	MSE	MAE
IC_tree	0.78	245.688	9.951	0.77	Inf	Inf	0.734	Inf	Inf
ctree	0.808	189.805	8.709	0.805	Inf	Inf	0.788	Inf	Inf
Middle									
regression	0.786	185.268	8.7	0.784	191.427	8.561	0.777	237.152	9.206
bagging	0.845	153.228	7.824	0.842	174.021	7.931	0.834	224.606	8.727
randomforest	0.786	148.457	7.732	0.784	176.396	7.945	0.777	227.581	8.764
boosting	0.817	184.306	9.347	0.822	182.478	8.655	0.818	227.259	9.132
xgboost	0.829	185.241	8.689	0.827	185.509	8.414	0.817	231.314	9.058
BART	0.851	143.184	7.831	0.856	166.972	7.798	0.852	215.728	8.542
ctree	0.817	183.672	8.62	0.813	191.153	8.46	0.804	233.925	9.064
rpart	0.804	190.124	8.73	0.8	Inf	Inf	0.782	Inf	Inf
XGB-GAM									
regression	0.787	184.85	8.706	0.784	191.859	8.582	0.776	237.183	9.213
bagging	0.847	152.633	7.808	0.844	173.824	7.912	0.835	224.664	8.723
randomforest	0.787	148.306	7.743	0.784	176.449	7.946	0.776	227.692	8.771
boosting	0.819	181.425	9.214	0.824	181.376	8.559	0.82	226.465	9.081
xgboost	0.832	182.786	8.597	0.83	184.018	8.307	0.819	230.683	9.009
BART	0.851	143.323	7.837	0.856	167.095	7.8	0.851	216.037	8.554
ctree	0.819	182.85	8.604	0.816	191.016	8.441	0.806	233.983	9.053
rpart	0.814	188.815	8.582	0.812	Inf	Inf	0.795	Inf	Inf

Table 5.25: Weibull model for Small Interval with n=200.

Method	no-censored			light-censored			heavy-censored		
	C-index	MSE	MAE	C-index	MSE	MAE	C-index	MSE	MAE
IC_tree	0.692	20.537	2.881	0.713	Inf	Inf	0.699	Inf	Inf
ctree	0.753	17.021	2.513	0.772	Inf	Inf	0.755	Inf	Inf
Middle									
regression	0.807	8.746	1.795	0.845	14.323	1.9	0.821	24.585	2.525
bagging	0.891	4.917	1.177	0.908	13.274	1.647	0.878	24.153	2.418
randomforest	0.807	6.105	1.419	0.845	14.51	1.805	0.821	24.795	2.49
boosting	0.843	6.204	1.575	0.881	13.7	1.776	0.85	24.354	2.519
xgboost	0.899	4.25	1.072	0.901	13.248	1.643	0.863	24.227	2.437
BART	0.82	7.549	1.802	0.838	17.722	2.375	0.832	26.996	2.915
ctree	0.752	16.784	2.537	0.781	18.408	2.375	0.733	27.742	2.91
rpart	0.845	9.391	1.751	0.833	Inf	Inf	0.813	Inf	Inf
XGB-GAM									
regression	0.807	8.649	1.791	0.845	14.315	1.9	0.822	24.583	2.528
bagging	0.89	4.947	1.187	0.908	13.297	1.654	0.877	24.158	2.421
randomforest	0.807	6.167	1.435	0.845	14.544	1.816	0.822	24.801	2.494
boosting	0.843	6.231	1.578	0.881	13.695	1.776	0.85	24.38	2.522
xgboost	0.899	4.298	1.075	0.901	13.25	1.643	0.863	24.22	2.437
BART	0.821	7.458	1.794	0.838	17.652	2.367	0.832	26.984	2.913
ctree	0.752	16.726	2.534	0.781	18.39	2.375	0.731	27.781	2.914
rpart	0.847	9.523	1.748	0.836	Inf	Inf	0.817	Inf	Inf

Table 5.26: Weibull model for Small Interval with n=300.

Method	no-censored			light-censored			heavy-censored		
	C-index	MSE	MAE	C-index	MSE	MAE	C-index	MSE	MAE
IC_tree	0.723	17.14	2.598	0.728	Inf	Inf	0.722	Inf	Inf
ctree	0.802	12.259	2.041	0.811	Inf	Inf	0.8	Inf	Inf
Middle									
regression	0.818	6.725	1.629	0.849	14.237	1.847	0.824	24.309	2.492
bagging	0.91	3.319	0.945	0.912	13.156	1.578	0.876	23.846	2.387
randomforest	0.818	4.293	1.14	0.849	13.982	1.683	0.824	24.275	2.432
boosting	0.87	4.472	1.299	0.896	13.419	1.666	0.859	23.983	2.44
xgboost	0.911	2.891	0.904	0.905	13.212	1.599	0.864	23.912	2.411
BART	0.838	5.003	1.471	0.85	15.451	2.034	0.832	25.86	2.78
ctree	0.803	11.675	2.029	0.817	16.462	2.109	0.78	25.83	2.683
rpart	0.855	8.815	1.669	0.843	Inf	Inf	0.818	Inf	Inf
XGB-GAM									
regression	0.817	6.787	1.638	0.849	14.217	1.848	0.825	24.308	2.493
bagging	0.909	3.331	0.951	0.912	13.166	1.581	0.876	23.851	2.389
randomforest	0.817	4.332	1.151	0.849	14.002	1.692	0.825	24.281	2.436
boosting	0.87	4.531	1.303	0.896	13.414	1.664	0.86	23.98	2.438
xgboost	0.911	2.922	0.909	0.905	13.211	1.598	0.864	23.912	2.409
BART	0.838	5.018	1.475	0.85	15.43	2.035	0.832	25.832	2.776
ctree	0.803	11.703	2.033	0.817	16.502	2.114	0.781	25.831	2.683
rpart	0.856	8.99	1.681	0.846	Inf	Inf	0.824	Inf	Inf

Table 5.27: Weibull model for Medium Interval with n=200.

Method	no-censored			light-censored			heavy-censored		
	C-index	MSE	MAE	C-index	MSE	MAE	C-index	MSE	MAE
IC_tree	0.692	269.852	11.19	0.707	Inf	Inf	0.708	Inf	Inf
ctree	0.743	232.814	9.931	0.764	Inf	Inf	0.751	Inf	Inf
Middle									
regression	0.827	95.956	6.425	0.852	143.68	6.588	0.831	256.92	8.511
bagging	0.917	46.938	3.742	0.933	124.051	4.99	0.896	247.887	7.689
randomforest	0.827	61.792	4.822	0.852	141.971	5.86	0.831	257.726	8.105
boosting	0.86	65.745	5.477	0.893	133.936	5.936	0.863	254.785	8.379
xgboost	0.928	36.557	3.15	0.931	120.39	4.74	0.884	247.809	7.688
BART	0.82	109.064	7.431	0.836	204.124	8.944	0.832	303.004	10.565
ctree	0.753	217.122	9.655	0.774	227.4	9.09	0.735	317.211	10.551
rpart	0.852	111.042	6.437	0.838	Inf	Inf	0.817	Inf	Inf
XGB-GAM									
regression	0.826	95.407	6.429	0.852	143.494	6.589	0.83	256.924	8.524
bagging	0.915	48.023	3.812	0.932	124.491	5.043	0.896	248.005	7.719
randomforest	0.826	63.084	4.912	0.852	142.688	5.935	0.83	257.913	8.145
boosting	0.861	65.8	5.482	0.892	134.548	5.979	0.862	255.134	8.396
xgboost	0.927	37.324	3.19	0.931	120.778	4.768	0.884	247.92	7.707
BART	0.821	106.989	7.368	0.835	204.133	8.947	0.832	302.936	10.563
ctree	0.752	219.196	9.702	0.776	226.164	9.057	0.734	317.59	10.572
rpart	0.854	113.79	6.492	0.845	Inf	Inf	0.824	Inf	Inf

Table 5.28: Weibull model for Medium Interval with n=300.

Method	no-censored			light-censored			heavy-censored		
	C-index	MSE	MAE	C-index	MSE	MAE	C-index	MSE	MAE
IC_tree	0.726	225.007	10.062	0.734	Inf	Inf	0.722	Inf	Inf
ctree	0.797	171.716	8.184	0.813	Inf	Inf	0.807	Inf	Inf
Middle									
regression	0.837	76.027	5.824	0.859	137.86	6.299	0.834	253.046	8.404
bagging	0.936	30.073	2.854	0.941	118.993	4.608	0.893	245.022	7.587
randomforest	0.837	41.754	3.742	0.859	130.702	5.241	0.834	250.898	7.844
boosting	0.891	45.743	4.357	0.916	124.948	5.238	0.88	247.595	7.931
xgboost	0.94	24.209	2.558	0.937	118.049	4.551	0.885	245.227	7.621
BART	0.843	66.835	5.735	0.852	162.99	7.376	0.836	281.2	9.828
ctree	0.808	156.366	7.809	0.824	181.459	7.621	0.793	281.314	9.354
rpart	0.865	95.178	5.941	0.85	Inf	Inf	0.82	Inf	Inf
XGB-GAM									
regression	0.837	77.306	5.884	0.859	137.499	6.309	0.833	252.867	8.414
bagging	0.935	30.964	2.922	0.94	119.256	4.657	0.893	245.116	7.621
randomforest	0.837	42.876	3.837	0.859	131.271	5.325	0.833	251.078	7.894
boosting	0.891	46.461	4.401	0.915	125.351	5.271	0.879	247.83	7.955
xgboost	0.94	24.469	2.609	0.937	118.131	4.564	0.885	245.37	7.642
BART	0.842	67.3	5.79	0.852	163.069	7.39	0.836	281.134	9.827
ctree	0.807	155.21	7.798	0.823	183.523	7.692	0.795	280.561	9.327
rpart	0.866	102.196	6.105	0.855	Inf	Inf	0.83	Inf	Inf

Table 5.29: Weibull model for Large Interval with n=200.

Method	no-censored			light-censored			heavy-censored		
	C-index	MSE	MAE	C-index	MSE	MAE	C-index	MSE	MAE
IC_tree	0.694	268.112	11.219	0.708	Inf	Inf	0.709	Inf	Inf
ctree	0.741	236.605	10.078	0.758	Inf	Inf	0.755	Inf	Inf
Middle									
regression	0.825	102.555	6.576	0.851	127.819	6.344	0.832	566.111	12.683
bagging	0.91	50.586	3.951	0.926	107.842	4.746	0.899	542.552	11.225
randomforest	0.825	64.165	4.866	0.851	124.756	5.493	0.832	565.351	11.814
boosting	0.859	68.713	5.582	0.888	119.552	5.762	0.868	558.628	12.302
xgboost	0.918	38.714	3.425	0.919	105.121	4.66	0.885	543.616	11.292
BART	0.818	113.176	7.559	0.829	195.602	8.941	0.836	685.18	15.934
ctree	0.753	218.98	9.713	0.771	218.766	9.085	0.742	725.296	16.005
rpart	0.846	117.14	6.687	0.832	Inf	Inf	0.816	Inf	Inf
XGB-GAM									
regression	0.826	102.98	6.599	0.85	127.809	6.377	0.832	565.655	12.7
bagging	0.909	52.674	4.065	0.924	108.451	4.831	0.898	542.967	11.293
randomforest	0.826	66.951	5.027	0.85	125.823	5.629	0.832	565.951	11.905
boosting	0.858	69.489	5.621	0.886	120.279	5.817	0.866	559.413	12.368
xgboost	0.918	39.344	3.468	0.919	105.384	4.691	0.884	544.744	11.346
BART	0.819	111.072	7.472	0.827	196.338	8.97	0.835	686.361	15.956
ctree	0.754	216.877	9.667	0.769	219.51	9.14	0.741	726.314	16.051
rpart	0.85	123.656	6.794	0.843	Inf	Inf	0.826	Inf	Inf

Table 5.30: Weibull model for Large Interval with n=300.

Method	no-censored			light-censored			heavy-censored		
	C-index	MSE	MAE	C-index	MSE	MAE	C-index	MSE	MAE
IC_tree	0.722	229.689	10.167	0.732	Inf	Inf	0.734	Inf	Inf
ctree	0.788	177.174	8.378	0.806	Inf	Inf	0.814	Inf	Inf
Middle									
regression	0.839	77.625	5.864	0.859	117.75	5.98	0.834	552.322	12.445
bagging	0.93	31.338	3.005	0.934	99.829	4.355	0.896	530.772	11.018
randomforest	0.839	42.756	3.76	0.859	110.582	4.847	0.834	544.703	11.373
boosting	0.889	47.713	4.465	0.911	106.81	5.029	0.882	537.256	11.567
xgboost	0.93	26.073	2.812	0.924	99.722	4.442	0.884	532.614	11.139
BART	0.843	68.593	5.785	0.85	148.004	7.187	0.84	617.788	14.432
ctree	0.799	165.137	8.061	0.823	166.642	7.481	0.805	615.362	13.741
rpart	0.859	95.607	6.029	0.84	Inf	Inf	0.82	Inf	Inf
XGB-GAM									
regression	0.836	80.145	5.959	0.857	117.726	6.024	0.832	552.272	12.489
bagging	0.927	33.336	3.136	0.932	100.357	4.445	0.895	531.141	11.092
randomforest	0.836	45.38	3.959	0.857	111.558	4.998	0.832	545.15	11.476
boosting	0.887	49.344	4.549	0.909	107.288	5.083	0.881	537.485	11.598
xgboost	0.93	27.353	2.881	0.925	100.117	4.473	0.885	532.687	11.172
BART	0.841	70.115	5.88	0.849	148.415	7.219	0.84	616.509	14.384
ctree	0.797	165.657	8.077	0.82	167.56	7.534	0.808	609.476	13.637
rpart	0.863	104.59	6.203	0.854	Inf	Inf	0.832	Inf	Inf

Table 5.31: Method selection under no censoring.

Model Interval	best method			second method			third method		
	Correct	Selected	rate%	Correct	Selected	rate%	Correct	Selected	rate%
linear-small	BART	BART	68.4	bagging	bagging	14	rf	rf	13.6
boxcox-small	BART	BART	67.8	bagging	xgb	13.8	xgb	bagging	12.8
ph-small	BART	BART	79.4	bagging	xgb	9.8	rf	bagging	7.4
aft-small	BART	BART	52.6	rf	rf	22.2	bagging	bagging	12.8
weibull-small	xgb	xgb	67.2	bagging	bagging	11.8	rf	rf	8.8
linear-medium	BART	BART	95.2	xgb	xgb	3	boosting	bagging	1.4
boxcox-medium	BART	BART	95.8	boosting	xgb	1.8	xgb	rf	1
ph-medium	BART	BART	95.2	xgb	xgb	4.4	boosting	boosting	0.2
aft-medium	BART	BART	50.6	rf	rf	22.2	bagging	bagging	14.4
weibull-medium	xgb	xgb	71.8	bagging	bagging	13.4	rf	boosting	4.8
linear-large	BART	BART	92	xgb	xgb	6.4	boosting	boosting	1.2
boxcox-large	BART	BART	95.2	xgb	xgb	2.6	boosting	bagging	1.2
ph-large	BART	BART	91.4	xgb	xgb	8	boosting	boosting	0.4
aft-large	BART	BART	35.2	rf	rf	26.2	bagging	bagging	19.6
weibull-large	xgb	xgb	71	bagging	bagging	13.2	rf	rf	5.4

Table 5.32: Method selection under light censoring

Model Interval	best method			second method			third method		
	Correct	Selected	rate%	Correct	Selected	rate%	Correct	Selected	rate%
linear-small	BART	BART	73.8	rf	bagging	12.6	bagging	rf	8.8
boxcox-small	BART	BART	85.4	xgb	bagging	6.6	bagging	xgb	5.8
ph-small	BART	BART	79.4	bagging	xgb	6	xgb	bagging	7.4
aft-small	BART	BART	71	bagging	rf	11	rf	bagging	9.8
weibull-small	xgb	xgb	66.2	bagging	bagging	32.6	boosting	boosting	1
linear-medium	BART	BART	91.4	rf	bagging	3	xgb	xgb	2.6
boxcox-medium	BART	BART	98	boosting	xgb	1.2	xgb	boosting	0.4
ph-medium	BART	BART	92	rf	xgb	6	xgb	boosting	1
aft-medium	BART	BART	68	bagging	rf	13.6	rf	bagging	10.8
weibull-medium	xgb	xgb	87.4	bagging	bagging	11	boosting	boosting	1
linear-large	BART	BART	92.2	rf	xgb	5.4	xgb	boosting	1.2
boxcox-large	BART	BART	95.6	boosting	xgb	3.4	xgb	rf	0.4
ph-large	BART	BART	90.2	rf	xgb	7.4	xgb	bagging	1.2
aft-large	BART	BART	53.2	bagging	rf	18	rf	bagging	17.2
weibull-large	xgb	xgb	87.8	bagging	bagging	10.6	boosting	rf	1

Table 5.33: Method selection under heavy censoring

Model Interval	best method			second method			third method		
	Correct	Selected	rate%	Correct	Selected	rate%	Correct	Selected	rate%
linear-small	BART	BART	64.6	bagging	bagging	14.4	rf	rf	10.2
boxcox-small	BART	BART	78.2	xgb	xgb	8.8	bagging	bagging	8
ph-small	BART	BART	70	boosting	bagging	13	xgb	rf	10.6
aft-small	BART	BART	61.8	bagging	rf	14.4	rf	bagging	12
weibull-small	bagging	bagging	46.2	xgb	xgb	43.2	boosting	boosting	5.6
linear-medium	BART	BART	79.8	boosting	xgb	7	rf	bagging	5.2
boxcox-medium	BART	BART	92.6	boosting	xgb	4.6	xgb	bagging	1.6
ph-medium	BART	BART	76	xgb	xgb	11.6	boosting	boosting	5.6
aft-medium	BART	BART	57.6	bagging	rf	17.4	xgb	bagging	14
weibull-medium	xgb	xgb	74.6	bagging	bagging	21.8	boosting	rf	2.2
linear-large	BART	BART	90.2	rf	xgb	6.2	xgb	boosting	1.6
boxcox-large	BART	BART	92	boosting	xgb	4	xgb	boosting	2.2
ph-large	BART	BART	86.4	xgb	xgb	9.2	rf	boosting	2
aft-large	BART	BART	52.6	bagging	bagging	21.8	boosting	rf	12.8
weibull-large	xgb	xgb	72.6	bagging	bagging	23	boosting	rf	3

Chapter 6

Real Data Analysis

In this chapter, we conduct a case study using interval-censored data on the eruption time of permanent teeth in elementary school students. The goal is to evaluate the reasonableness of predicted eruption times across various methods, especially when prediction is not feasible through traditional time-to-event analysis. We also aim to compare different model performances under multiple settings and summarize our findings and conclusions.

6.1 Data Description

We use the `tandmob2` dataset provided in the R package `bayesSurv`. This dataset was originally used in the paper *Survival trees for interval-censored survival data* by Fu and Simonoff (2017), and contains exactly the same data used in that study. The data originate from a longitudinal dental health study conducted in Copenhagen, Denmark between 1996 and 2001, comprises a total of 4790 interval-censored observations on tooth eruption time. The right censired rate in this dataset is approximately 24%, and we assign 70% of the data as the training set and the

remaining 30% as the test set. In Lesaffre et al. (2005)'s work, the authors employed a proportional hazards model to estimate the eruption time of permanent teeth, based on variables such as sugar consumption, dietary habits, and oral hygiene practices. The outcome of interest was the interval during which the first permanent molar erupted (referred to as tooth number 26 in the FDI system). In our study, we selected 8 predictor variables considered relevant to the eruption time of permanent teeth. These include: `gender` (0 for male, 1 for female), `province`, `education system type`, `starbr` is starting age of brushing the teeth, `fluor` is a binary variable indicating the presence of dental fluorosis (1 = yes, 0 = no), age at first dental examination, number of extracted primary teeth (`badtooth`), number of decayed or filled primary teeth (`dtooth`), and the number of primary teeth extracted due to orthodontic treatment or decay at the examination prior to the eruption of permanent teeth (`ctooth`).

6.2 Plausibility evaluation

In this analysis, the BART model was selected according selection 4.3. The rule of this evaluation is to measure the proportion of predicted values that successfully fall within the correct (L_i, R_i) interval for each method. The assessment is conducted on the entire test set, regardless of whether the data is right-censored. The results

Table 6.1: Plausibility test

Model	ICtree	ctree	XGB-GAM	Middle
rate%	33.67	34.68	35.91	36.47

show that our two proposed methods achieve the highest proportion, though not remarkably superior. A possible reason is that the interval lengths in the dataset

are mostly less than 1, making it difficult for any method to accurately predict values within such narrow ranges. The results of the other two methods also show that the proportions are not particularly high.

6.3 Range of Predicted Times

Since the target of prediction is tooth eruption time, a normally distributed pattern would be the most reasonable expectation. As shown in Figure 6.1, the x-axis represents different imputation methods as well as ICtree and ctree, while the y-axis indicates the predicted eruption time, the IC tree and Ctree methods do not particularly conform to a normal distribution. The IC tree displays a clear right-skewed distribution, and its predicted times are overly conservative, mostly concentrated between 10 and 11. On the other hand, the Ctree method exhibits a left-skewed distribution, with a slightly wider prediction range than the IC tree, but still largely restricted between 10 and 11. These characteristics deviate from the typical properties of a normal distribution.In contrast, the two proposed methods yield very similar results that appear much closer to a normal distribution in the boxplot representation. Moreover, the predicted time range is noticeably wider, which aligns with our intended modeling objective

6.4 Predicted Times across Different Subgroups

The BART model was selected from section 4.3. The y-axis represents the proportion corresponding to each age interval, while the x-axis indicates the predicted eruption time. The length of age interval is 0.15years. Figures 6.2 to 6.5 present the predicted eruption times stratified by gender. These plots clearly show that boys

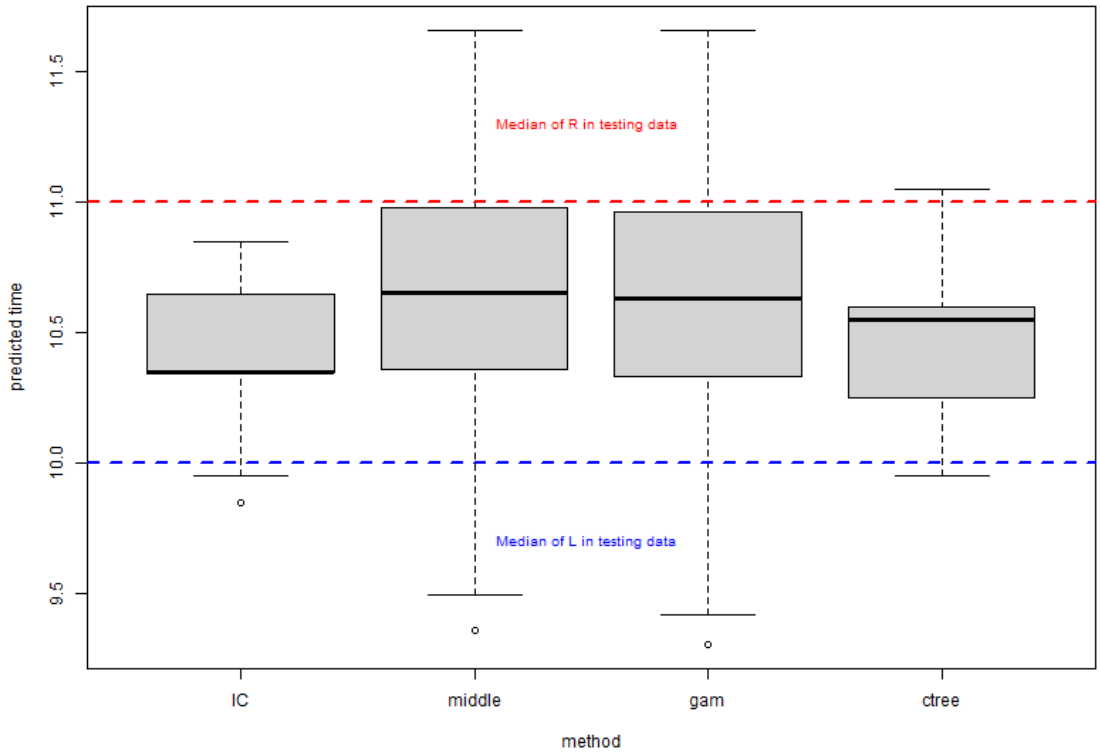

Figure 6.1: The boxplot of predicted time for the different methods

tend to have later tooth eruption times compared to girls, which aligns with common developmental knowledge, as boys generally mature more slowly than girls. However, from a visual perspective, some notable differences can be observed among the methods. Specifically, both the IC tree and Ctree models exhibit a common issue: their estimated curves do not decline toward the end of the time range. This may be attributed to the way tree-based methods partition the data and treat right-censored observations as belonging to a fewer groups, which in turn weakens the ability to depict a clear trend. In contrast, our proposed methods preserve a more evident trend across time, offering better visual interpretability for researchers. Figures 6.6 to 6.9 illustrate predicted eruption times grouped by type of educational institution. Based on the visual results, there appears to be no significant difference in eruption timing across different education types. Nonetheless, our two proposed methods again produce results that more closely resemble a normal distribution. It is worth

noting that for the "free" education type, the estimated eruption time at the endpoint is slightly higher than that of the other two categories. This difference is likely due to a larger number of observations in the "free" group, rather than a genuine effect of education type. Figures 6.10 to 6.13 display predicted eruption times based on whether the individual used fluoride toothpaste. Interestingly, those identified as having fluoride exposure show slightly delayed eruption times compared to those without fluoride intake. After checking the gender distribution across groups, we confirmed that this was not due to gender imbalance. Although this result contradicts general expectations, several explanations are possible. First, sample size differences: the non-fluoride group has significantly more observations, which may introduce estimation bias. Second, according to the conclusion in Iamandii et al. (2024), exposure to high-fluoride drinking water appears to non-linearly affect thyroid function, increasing TSH secretion and the risk of certain thyroid disorders in children. Some existing studies suggest that thyroid disorders, particularly hypothyroidism, are positively associated with both children's and adults' impaired thyroid function. Vucic et al. (2017) further observed that children with low thyroid hormone levels (i.e., hypothyroidism) tend to exhibit delayed tooth eruption and enamel hypoplasia, while children with thyroid disorders may experience decreased thyroid function, which can in turn lead to delayed eruption of permanent teeth.. Taken together, although there is currently no direct empirical evidence linking fluoride exposure to changes in eruption timing, the combination of these two studies suggests a plausible physiological mechanism. Excessive fluoride exposure may affect thyroid function, which in turn could interfere with dental development. This hypothesis is biologically reasonable and warrants further investigation.

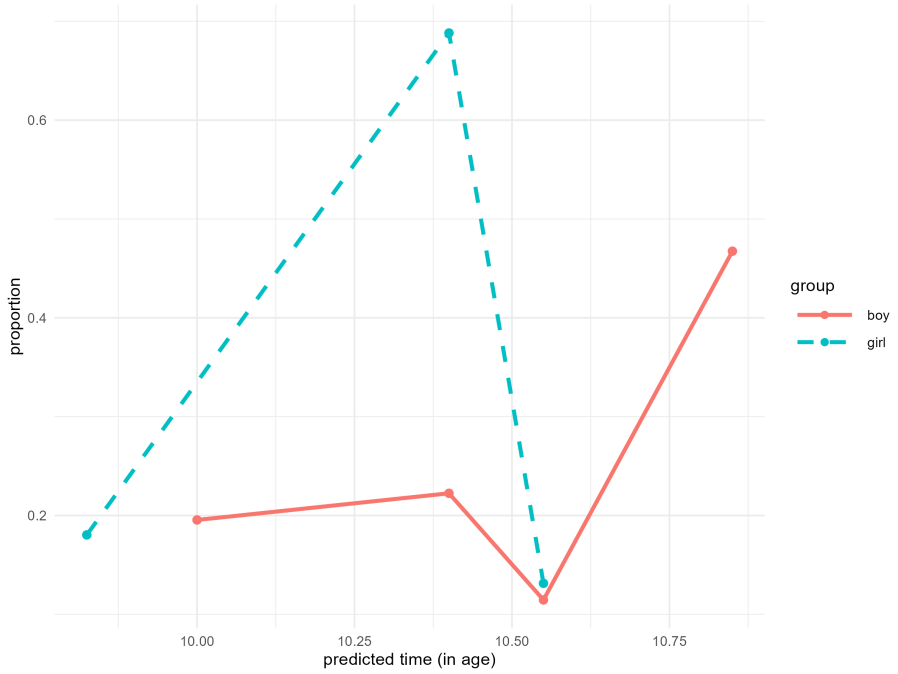

Figure 6.2: The proportion versus the predicted time of IC Tree Classified by Gender.

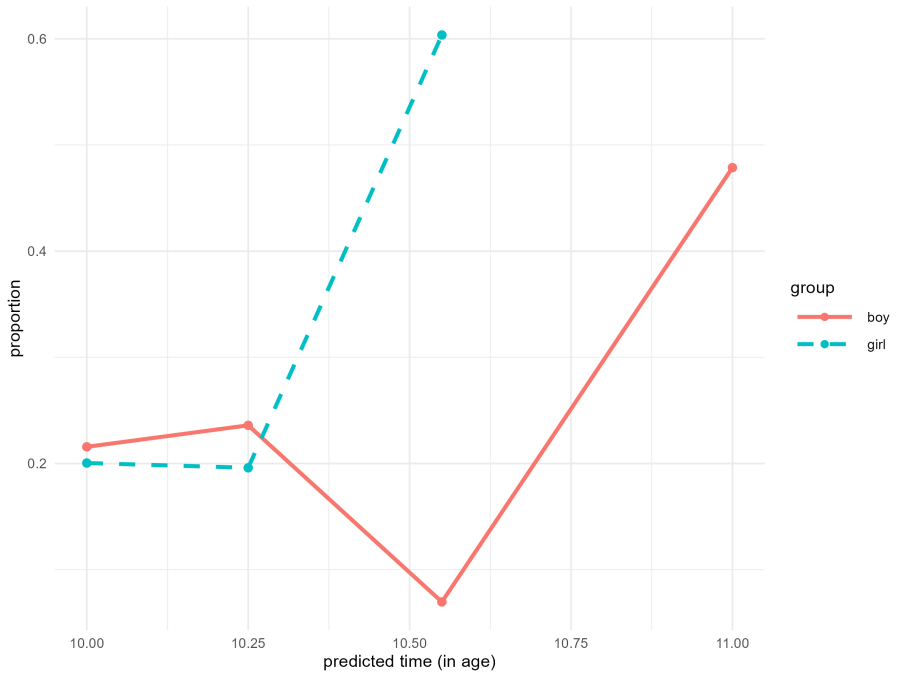

Figure 6.3: The proportion versus the predicted time of Ctree Classified by Gender.

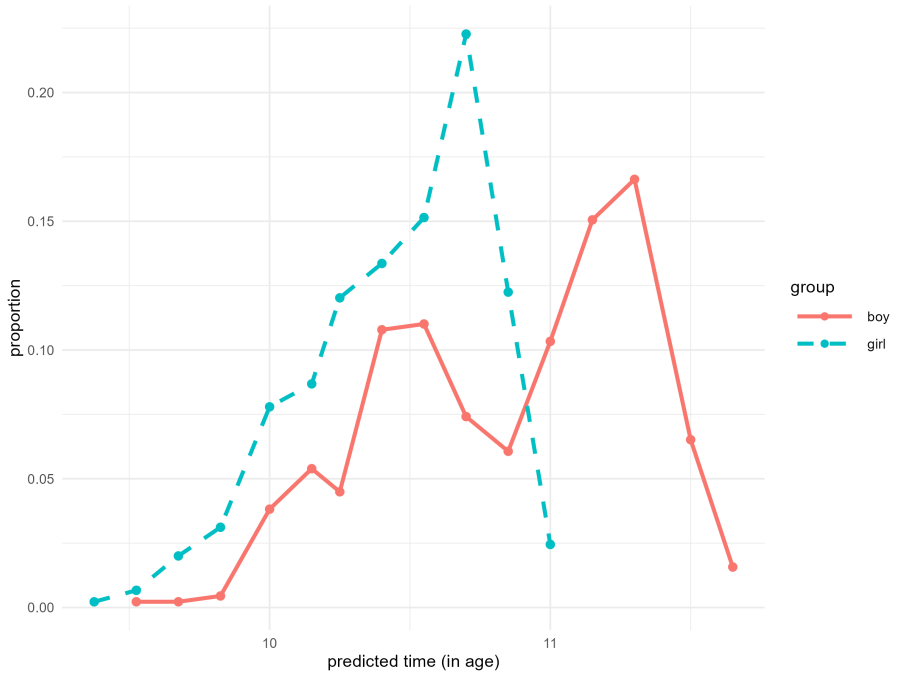

Figure 6.4: The proportion versus the predicted time of Middle Imputation Classified by Gender.

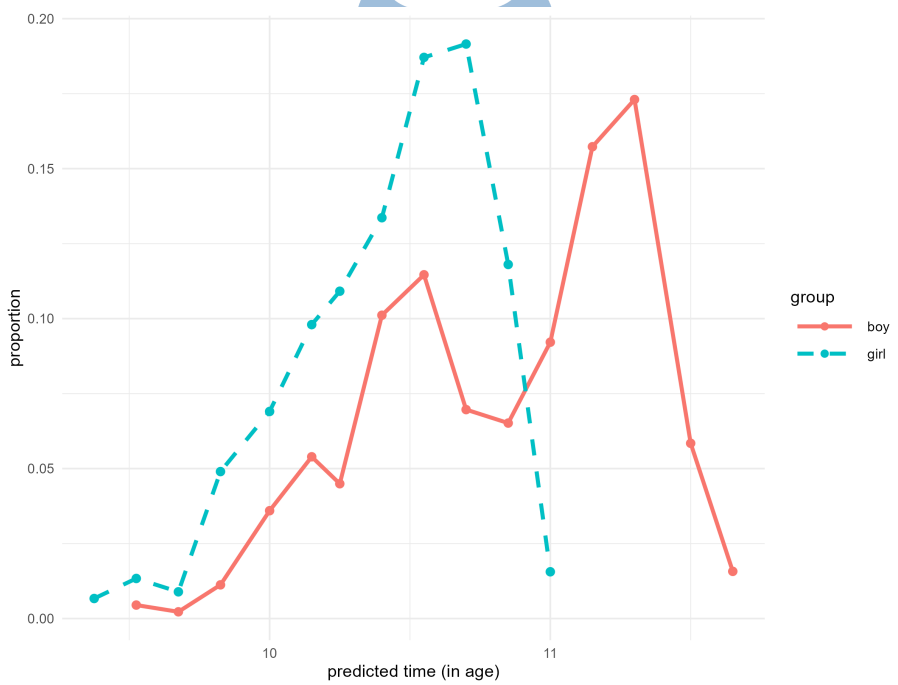

Figure 6.5: The proportion versus the predicted time of XGB-GAM Imputation Classified by Gender.

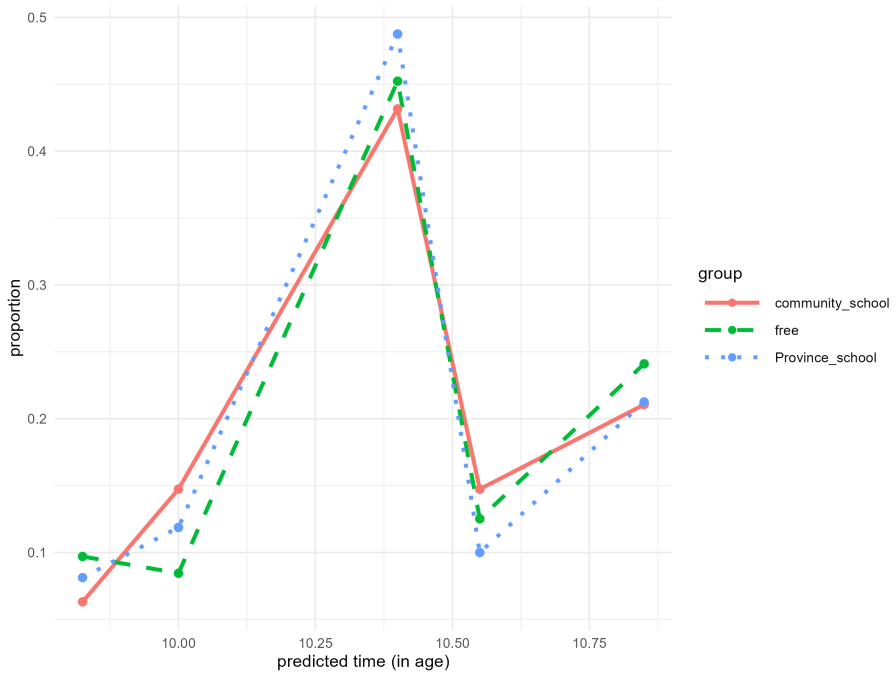

Figure 6.6: The proportion versus the predicted time of IC Tree Classified by Education.

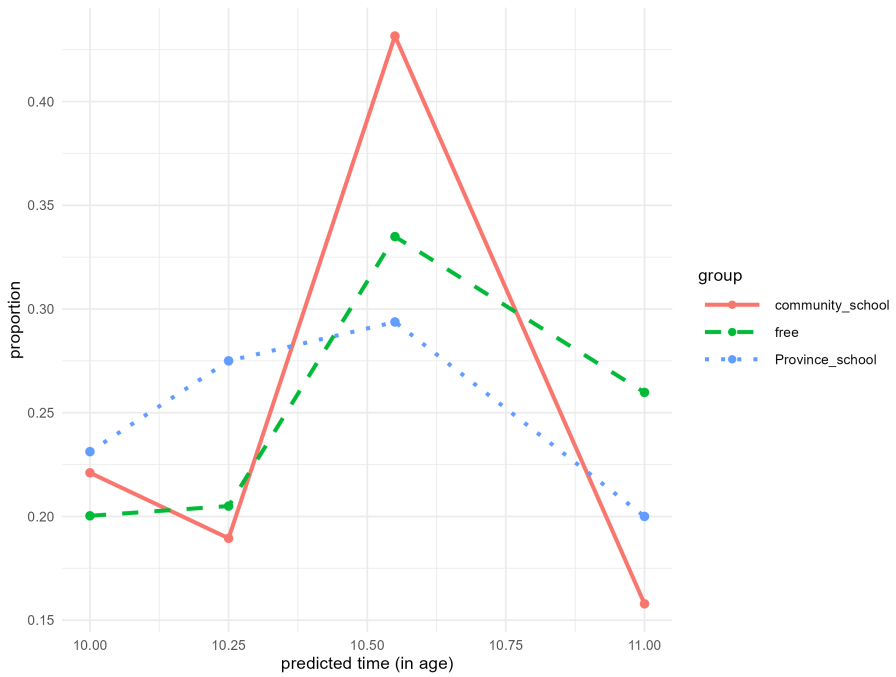

Figure 6.7: The proportion versus the predicted time of Ctree Classified by Education.

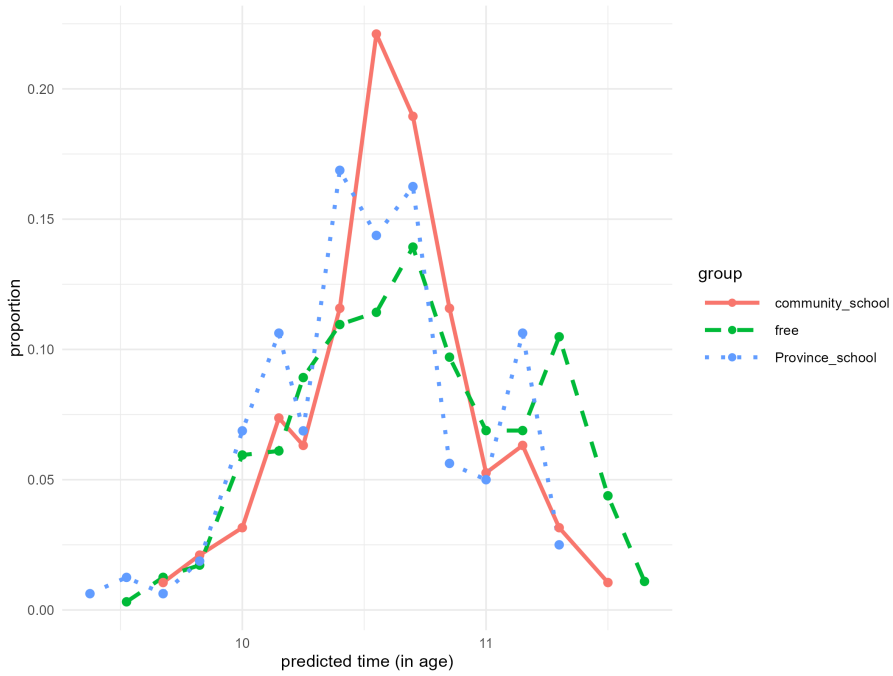

Figure 6.8: The proportion versus the predicted time of Middle Imputation Classified by Education.

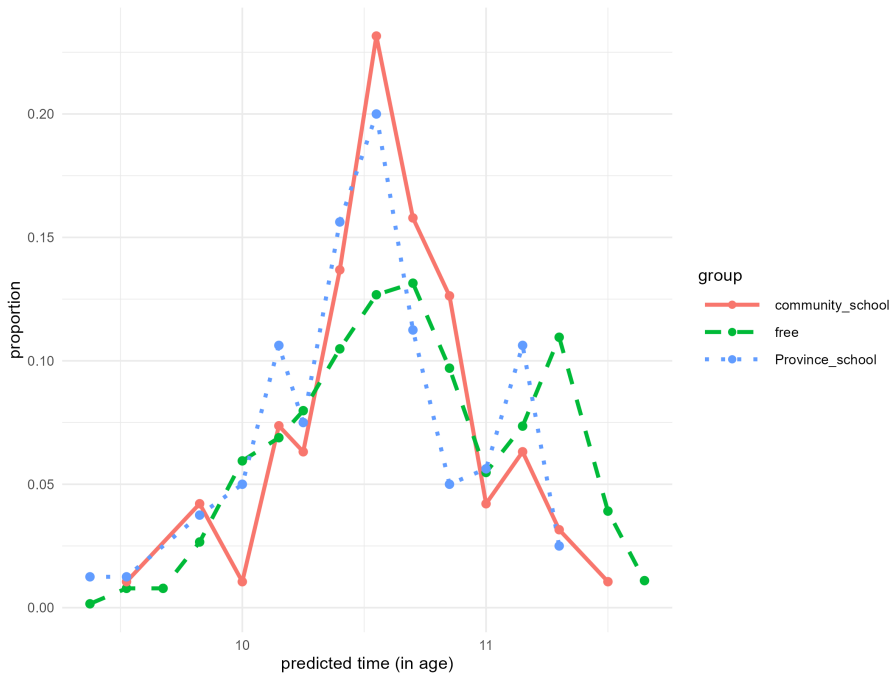

Figure 6.9: The proportion versus the predicted time of XGB-GAM Imputation Classified by Education.

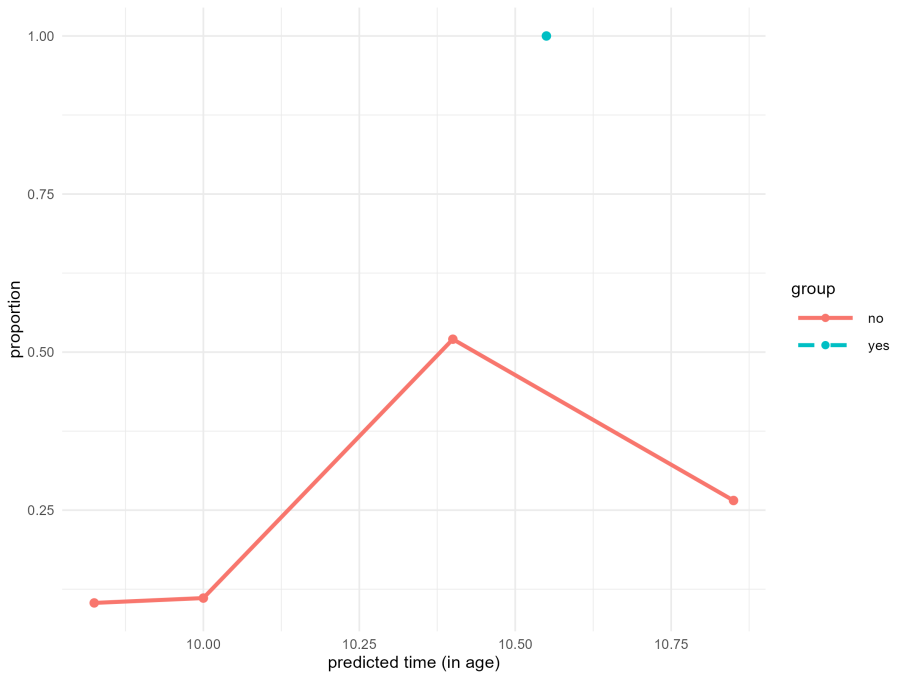

Figure 6.10: The proportion versus the predicted time of IC Tree Classified by Fluor.

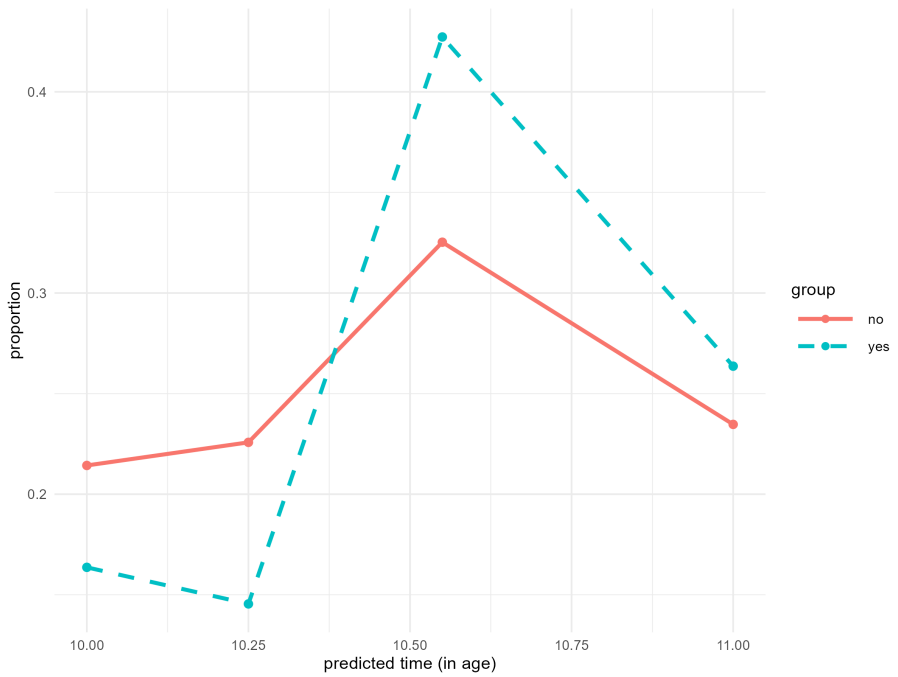

Figure 6.11: The proportion versus the predicted time of Ctree Classified by Fluor.

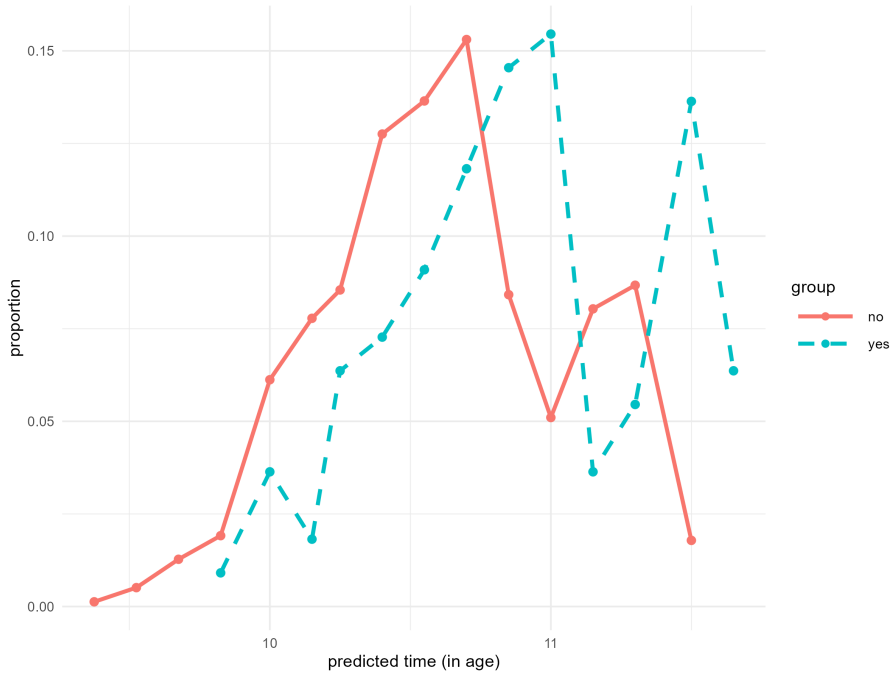

Figure 6.12: The proportion versus the predicted time of Middle Imputation Classified by Fluor.

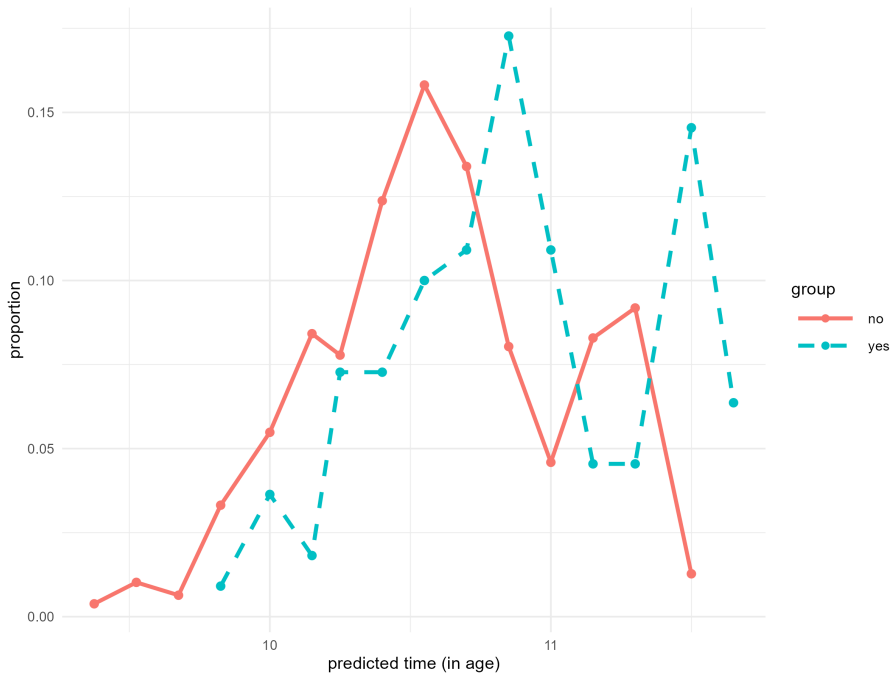

Figure 6.13: The proportion versus the predicted time of XGB-GAM Imputation Classified by Fluor.

Chapter 7

Conclusion

In this study, we first introduced the concept of interval-censored data and the Generalized Accelerated Failure Time (GAFT) model, emphasizing the need for imputation in such contexts. The most intuitive approach we initially considered was midpoint imputation. Subsequently, we developed an iterative method that combines XGBoost (XGB) with Generalized Additive Models (GAM) to estimate imputed values for interval-censored data. Additionally, through multiple experiments, we found that assuming a Gamma distribution for right-censored data yielded the best estimation performance. By integrating the imputed values from both interval- and right-censored data, we applied various tree-based models to predict survival times. Through the simulation experiments in Chapter 5 and the real-world case study in Chapter 6, we demonstrated that both our proposed XGB-GAM combined with Gamma-imputed right-censoring method and the midpoint method combined with Gamma-imputed right-censoring outperformed the existing ICtree approach. Since different situations may require different tree models for optimal performance, we also developed a model selection mechanism that can effectively identify the most suitable tree model for survival time prediction. In the real-data application, this mechanism successfully selected BART, the most competitive model—a result

that was consistent with findings from the simulation study. Although objective metrics such as MSE cannot be directly applied in the real data scenario to evaluate effectiveness, our methods produced estimated survival times that were more normally distributed, with reasonable range and less tendency for predictions to cluster around a few tree. Between the two methods—XGB-GAM + Gamma and Midpoint + Gamma—although both showed comparable performance in terms of C-index, MSE, and MAE during simulations, the midpoint method combined with Gamma-based right-censoring imputation exhibited clear advantages in terms of computational time and complexity. Therefore, we recommend this latter approach as the preferred method for practical applications.

Reference

[1] Anderson-Bergman, C. (2017). icenReg: Regression models for interval censored data in R. *Journal of Statistical Software, 81*(12), 1–23.

[2] Breiman, L. (1996). Bagging predictors. *Machine Learning, 24*(2), 123–140.

[3] Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5–32.

[4] Breiman, L., Friedman, J., Olshen, R. A., & Stone, C. J. (1984). *Classification and regression trees.* CRC Press.

[5] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794).

[6] Chipman, H. A., George, E. I., & McCulloch, R. E. (2010). BART: Bayesian additive regression trees. *The Annals of Applied Statistics, 4*(1), 266–298.

[7] Ciampi, A., Hogg, S., & Thiffault, J. (1987). Recursive partition: A versatile method for exploratory data analysis in biostatistics. In K. M. E. Halperin & M. J. Feinstein (Eds.), *Modern statistical methods in chronic disease epidemiology* (pp. 191–204).

[8] Cox, D. R. (1972). Regression models and life-tables. *Journal of the Royal Statistical Society: Series B (Methodological), 34*(2), 187–220.

[9] Delignette-Muller, M. L., & Dutang, C. (2015). fitdistrplus: An R package for fitting distributions. *Journal of Statistical Software, 64*(4), 1–34.

[10] Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *Annals of Statistics, 29*(5), 1189–1232.

[11] Fu, W., & Simonoff, J. S. (2017). Survival trees for interval-censored survival data. *Statistics in Medicine, 36*(30), 4831–4842.

[12] Gordon, L., & Olshen, R. A. (1985). Tree-structured survival analysis. *Cancer Treatment Reports, 69*(10), 1065–1069.

[13] Han, A. K. (1987). Non-parametric analysis of a generalized regression model: The maximum rank correlation estimator. *Journal of Econometrics, 35*(2–3), 303–316.

[14] Harrell, F. E., Jr., Califf, R. M., Pryor, D. B., Lee, K. L., & Rosati, R. A. (1982). Evaluating the yield of medical tests. *Journal of the American Medical Association, 247(18), 2543–2546.*(18), 2543–2546.

[15] Hothorn, T., Hornik, K., & Zeileis, A. (2006). Unbiased recursive partitioning: A conditional inference framework. *Journal of Computational and Graphical Statistics, 15*(3), 651–674.

[16] Hothorn, T., Hornik, K., & Zeileis, A. (2010). *party: A laboratory for recursive partitioning.* R package version.

[17] Iamandii, I., De Pasquale, L., Giannone, M. E., Veneri, F., Generali, L., Consolo, U., Birnbaum, L. S., Castenmiller, J., Halldorsson, T. I., Filippini, T., & Vinceti, M. (2024). Does fluoride exposure affect thyroid function? A systematic review and dose-response meta-analysis. *Environmental Research, 242*, Article 117759.

[18] Kalbfleisch, J. D., & Prentice, R. L. (1980). *The statistical analysis of failure time data.* Wiley.

[19] Komárek, A., Lesaffre, E., Härkänen, T., Declerck, D., & Virtanen, J. I. (2005). A Bayesian analysis of multivariate doubly-interval-censored dental data. *Biostatistics, 6*(1), 145–155.

[20] Molinaro, A. M., Dudoit, S., & van der Laan, M. J. (2004). Tree-based multivariate regression and density estimation with right-censored data. *Journal of Multivariate Analysis, 90*(1), 154–177.

[21] Morgan, J. N., & Sonquist, J. A. (1963). Problems in the analysis of survey data, and a proposal. *Journal of the American Statistical Association, 58*(302), 415–434.

[22] Ridder, G. (1990). The non-parametric identification of generalized accelerated failure-time models. *The Review of Economic Studies, 57*(2), 167–181.

[23] Therneau, T. M., & Atkinson, B. (2010). *rpart: Recursive partitioning and regression trees.* R package version.

[24] Uno, H., Cai, T., Pencina, M. J., D'Agostino, R. B., & Wei, L. J. (2011). On the C-statistics for evaluating overall adequacy of risk prediction procedures with censored survival data. *Statistics in Medicine, 30*(10), 1105–1117.

[25] Vučić, S., Korevaar, T. I. M., Dhamo, B., Jaddoe, V. W. V., Peeters, R. P., Wolvius, E. B., & Ongkosuwito, E. M. (2017). Thyroid function during early life and dental development. *Journal of Dental Research, 96*(9), 1020–1026.

[26] Zhou, Y., & McArdle, J. J. (2015). Rationale and applications of survival tree and survival ensemble methods. *Psychometrika, 80*(3), 811–833.